Story Docs

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

📝 IPA Metadata Standard

> An overview of the IP-specific metadata standard

<Warning>
  We are still figuring out the best way to define an IPA Metadata Standard. For
  the sake of transparency, the following document is our thoughts so far but is
  subject to change as we release future versions.
</Warning>

<CardGroup cols={2}>
  <Card title="Official Ippy IP" href="https://explorer.story.foundation/ipa/0xB1D831271A68Db5c18c8F0B69327446f7C8D0A42" icon="house">
    Check out the official Ippy IP, which has both NFT & IP metadata.
  </Card>

  <Card title="How to Add Metadata to an IP Asset" href="/concepts/ip-asset/overview#nft-vs-ip-metadata" icon="computer">
    Learn how to actually add the IP metadata discussed here to your IP Asset with an explanation or completed code example.
  </Card>
</CardGroup>

This is the JSON metadata that is associated with an IP Asset, and gets stored inside of an IP Account. You must call `setMetadata(...)` inside of the IP Account in order to set the metadata, and then call `metadata()` to read it.

Attributes & Structure

Below are the important attributes you should provide in your IP metadata. Under the **Required For** column is what the specific field is required for:

* 🔍 Story Explorer - this field will help display your IP on the Story Explorer
* 🕵️ [Commercial Infringement Check](/concepts/story-attestation-service) - this field is required if your IP is **commercial** (that is, has `commercialUse = true` license terms attached). We will use these fields to run an infringement check on your IP.
  * See [current limitations](/concepts/story-attestation-service#current-limitations).
* 🤖 AI Agents - used for displaying metadata associated with AI Agents

Property Name	Type	Description	Required	For
`title`	`string`	Title of the IP	🔍 Story Explorer	
`description`	`string`	Description of the IP	🔍 Story Explorer	
`createdAt`	`string`	Date/Time that the IP was created (either ISO8601 or unix format). This field can be used to specify historical dates that aren't on-chain. For example, Harry Potter was published on June 26.	🔍 Story Explorer	
`image`	`string`	An image for your IP. **For audio assets, the recommended thumbnail aspect ratio is 1:1. For video assets, it is 16:9.**	🔍 Story Explorer	
`imageHash`	`string`	Hash of your `image` using SHA-256 hashing algorithm. See [here](#hashing-content) for how that is done.	🔍 Story Explorer	
`creators`	`IpCreator[]`	An array of information about the creators. [See the type defined below](#type-definitions)	🔍 Story Explorer	
`mediaUrl`	`string`	Used for infringement checking, points to the actual media (ex. image or audio). **For audio assets, the recommended thumbnail aspect ratio is 1:1. For video assets, it is 16:9.**	🕵️ [Commercial Infringement Check](/concepts/story-attestation-service)	
`mediaHash`	`string`	Hashed string of the media using SHA-256 hashing algorithm. See [here](#hashing-content) for how that is done.	🕵️ [Commercial Infringement Check](/concepts/story-attestation-service)	
`mediaType`	`string`	Type of media (audio, video, image), based on [mimeType](https://developer.mozilla.org/en-US/docs/Web/HTTP/MIME_types/Common_types). See the allowed media types [here](#media-types).	🕵️ [Commercial Infringement Check](/concepts/story-attestation-service)	
`aiMetadata`	`AIMetadata`	Used for registering & displaying AI Agent Metadata. [See the type defined below](#type-definitions)	🤖 AI Agents	
N/A	N/A	You can include other values as well.	N/A	

Type Definitions

Here are the type definitions for the complex types used in the metadata:

<CodeGroup>

```typescript IpCreator theme={null}
type IpCreator = {
  name: string;
  address: Address;
  contributionPercent: number; // add up to 100
  description?: string;
  image?: string;
  socialMedia?: IpCreatorSocial[];
  role?: string;
};
```

```typescript
type IpCreatorSocial = {
  platform: string;
  url: string;
};
```

```typescript AIMetadata theme={null}
type AIMetadata = {
  // this can be any character file you want
  // example: https://github.com/elizaOS/characterfile/blob/main/examples/example.character.json
  characterFileUrl: string;
  characterFileHash: string;
};
```

</CodeGroup>

Media Types

The following media types are allowed for the `mediaType` field:

Media Type	Description
`image/jpeg`	JPEG image
`image/png`	PNG image
`image/apng`	Animated PNG image
`image/avif`	AV1 Image File Format
`image/gif`	GIF image
`image/svg+xml`	SVG image
`image/webp`	WebP image
`audio/wav`	WAV audio
`audio/mpeg`	MP3 audio
`audio/flac`	FLAC audio
`audio/aac`	AAC audio
`audio/ogg`	OGG audio
`audio/mp4`	MP4 audio
`audio/x-aiff`	AIFF audio
`audio/x-ms-wma`	WMA audio
`audio/opus`	Opus audio
`video/mp4`	MP4 video
`video/webm`	WebM video
`video/quicktime`	QuickTime video

Hashing Content

To hash content for the `imageHash` or `mediaHash` fields, you can use the SHA-256 hashing algorithm. Here's an example of how to do this in JavaScript:

<CodeGroup>

```typescript TypeScript theme={null}
import { toHex, Hex } from "viem";

// get hash from a file
async function getFileHash(file: File): Promise<Hex> {
  const arrayBuffer = await file.arrayBuffer();
  const hashBuffer = await crypto.subtle.digest("SHA-256", arrayBuffer);
  return toHex(new Uint8Array(hashBuffer), { size: 32 });
}

// get hash from a url
async function getHashFromUrl(url: string): Promise<Hex> {
  const response = await axios.get(url, { responseType: "arraybuffer" });
  const buffer = Buffer.from(response.data);
  return "0x" + createHash("sha256").update(buffer).digest("hex");
}
```

```shell Shell theme={null}
shasum -a 256 myfile.jpg
```

</CodeGroup>

Example Use Cases

<Tabs>
  <Tab title="Ippy Mascot">
    This is the official Ippy mascot that is registered on mainnet. You can view it on our protocol explorer [here](https://explorer.story.foundation/ipa/0xB1D831271A68Db5c18c8F0B69327446f7C8D0A42).

    ```json  theme={null}
    {
      "title": "Ippy",
      "description": "Official mascot of Story.",
      "createdAt": "1728401700",
      "image": "https://ipfs.io/ipfs/QmSamy4zqP91X42k6wS7kLJQVzuYJuW2EN94couPaq82A8",
      "imageHash": "0x21937ba9d821cb0306c7f1a1a2cc5a257509f228ea6abccc9af1a67dd754af6e",
      "mediaUrl": "https://ipfs.io/ipfs/QmSamy4zqP91X42k6wS7kLJQVzuYJuW2EN94couPaq82A8",
      "mediaHash": "0x21937ba9d821cb0306c7f1a1a2cc5a257509f228ea6abccc9af1a67dd754af6e",
      "mediaType": "image/png",
      "creators": [
        {
          "name": "Story Foundation",
          "address": "0x67ee74EE04A0E6d14Ca6C27428B27F3EFd5CD084",
          "description": "The World's IP Blockchain",
          "contributionPercent": 100,
          "socialMedia": [
            {
              "platform": "Twitter",
              "url": "https://twitter.com/storyprotocol"
            },
            {
              "platform": "Telegram",
              "url": "https://t.me/StoryAnnouncements"
            },
            {
              "platform": "Website",
              "url": "https://story.foundation"
            },
            {
              "platform": "Discord",
              "url": "https://discord.gg/storyprotocol"
            },
            {
              "platform": "YouTube",
              "url": "https://youtube.com/@storyFDN"
            }
          ]
        }
      ],
      "tags": ["Ippy", "Story", "Story Mascot", "Mascot", "Official"], // experimental field
      "ipType": "Character" // experimental field
    }
    ```
  </Tab>

  <Tab title="Music">
    This is an example song generated on [Suno](https://suno.com/) and registered on our testnet. View the below example [on our protocol explorer](https://aeneid.explorer.story.foundation/ipa/0x7d126DB8bdD3bF88d757FC2e99BFE3d77a55509b).

    ```json  theme={null}
    {
      "title": "Midnight Marriage",
      "description": "This is a house-style song generated on suno.",
      "createdAt": "1740005219",
      "creators": [
    ```

```
      {
        "name": "Jacob Tucker",
        "address": "0xA2f9Cf1E40D7b03aB81e34BC50f0
A8c67B4e9112",
        "contributionPercent": 100
      }
    ],
    "image": "https://cdn2.suno.ai/image_large_8bcba6bc-3f60-
4921-b148-f32a59086a4c.jpeg",
    "imageHash": "0xc404730cdcdf7e5e54e8f16bc668
7f97c6578a296f4a21b452d8a6ecabd61bcc",
    "mediaUrl": "https://cdn1.suno.ai/dcd3076f-3aa5-400b-ba5d-
87d30f27c311.mp3",
    "mediaHash": "0xb52a44f53b2485ba772bd4857a44
3e1fb942cf5dda73c870e2d2238ecd607aee",
    "mediaType": "audio/mpeg"
  }
```


</Tab>

<Tab title="AI Agent">
The main difference here is you should supply `aiMetadata` with a character file. You can provide any character file you want, or use [this ElizaOS example](https://github.com/elizaOS/characterfile/blob/main/examples/example.character.json) as a template.

View the below example [on our protocol explorer](https://aeneid.explorer.story.foundation/ipa/0x49614De8b2b02C790708243F268Af50979D568d4).


```json  theme={null}
{
  "title": "Story AI Agent",
  "description": "This is an example AI Agent registered on Story.",
  "createdAt": "1740005219",
  "creators": [
    {
      "name": "Jacob Tucker",
      "address": "0xA2f9Cf1E40D7b03aB81e34BC50f0
A8c67B4e9112",
      "contributionPercent": 100
    }
  ],
  "image": "https://ipfs.io/ipfs/bafybeigi3k77t5h5aefwpzvx3uiom
uavdvqwn5rb5uhd7i7xcq466wvute",
  "imageHash": "0x64ccc40de203f218d16bb90878ec
ca4338e566ab329bf7be906493ce77b1551a",
  "mediaUrl": "https://ipfs.io/ipfs/
bafybeigi3k77t5h5aefwpzvx3uiom
uavdvqwn5rb5uhd7i7xcq466wvute",
  "mediaHash": "0x64ccc40de203f218d16bb90878ec
ca4338e566ab329bf7be906493ce77b1551a",
  "mediaType": "image/webp",
  "aiMetadata": {
    "characterFileUrl": "https://ipfs.io/ipfs/
bafkreic6eu4hlnwx46soib62rgkhh
mlieko67dggu6bzk7bvtfusqsknfu",
    "characterFileHash": "0x5e253875b6d7e7a4e407da899473
b168229def8cc6a783957c35996928494d2d"
  }
}
```


</Tab>
</Tabs>

Optional Properties

The following properties are optional but can provide additional context about your IP Asset:

<Warning>
We are still figuring out the best way to define an IPA Metadata Standard. The

fields below are bound to change or be removed at some point.
</Warning>

Property Name	Type	Description
`ipType`	`string`	Type of the IP Asset, can be defined arbitrarily by the creator. I.e. "character", "chapter", "location", "items", "music", etc
`relationships`	`IpRelationship[]`	The detailed relationship info with the IPA's direct parent asset, such as `APPEARS_IN`, `FINETUNED_FROM`, etc. See more examples [here](#relationship-types).
`watermarkImage`	`string`	A separate image with your watermark already applied. This way apps choosing to use it can render this version of the image (with watermark applied).
`media`	`IpMedia[]`	An array of supporting media. Media type defined below
`app`	`StoryApp`	This is assigned to verified application from Story directly (on a request basis so far). We will map each App ID to a name
`tags`	`string[]`	Any tags that can help surface this IPA
`robotTerms`	`IPRobotTerms`	Allows you to set Do Not Train for a specific agent
N/A	N/A	You can include other values as well.

Type Definitions

<CodeGroup>
```typescript IpRelationship theme={null}
type IpRelationship = {
  parentIpId: Address;
  type: string; // see "Relationship Types" docs below
};
```

```typescript IpMedia theme={null}
type IpMedia = {
  name: string;
  url: string;
  mimeType: string;
};
```

```typescript StoryApp theme={null}
type StoryApp = {
  id: string;
  name: string;
  website: string;
  action?: string;
};
```

```typescript IPRobotTerms theme={null}
type IPRobotTerms = {
  userAgent: string;
  allow: string;
};
```
</CodeGroup>

Relationship Types

The different relationship types that can be used for the `relationships` attribute.

Story Relationships

1. **APPEARS_IN** - A character APPEARS_IN a chapter.

2. **BELONGS_TO** - A chapter BELONGS_TO a book.

3. **PART_OF** - A book is PART_OF a series.

4. **CONTINUES_FROM** - A chapter CONTINUES_FROM the previous one.

5. **LEADS_TO** - An event LEADS_TO a consequence.

6. **FORESHADOWS** - An event FORESHADOWS future developments.

7. **CONFLICTS_WITH** - A character CONFLICTS_WITH another character.

8. **RESULTS_IN** - A decision RESULTS_IN a significant change.

9. **DEPENDS_ON** - A subplot DEPENDS_ON the main plot.

10. **SETS_UP** - A prologue SETS_UP the story.

11. **FOLLOWS_FROM** - A chapter FOLLOWS_FROM the previous one.

12. **REVEALS_THAT** - A twist REVEALS_THAT something unexpected occurred.

13. **DEVELOPS_OVER** - A character DEVELOPS_OVER the course of the story.

14. **INTRODUCES** - A chapter INTRODUCES a new character or element.

15. **RESOLVES_IN** - A conflict RESOLVES_IN a particular outcome.

16. **CONNECTS_TO** - A theme CONNECTS_TO the main narrative.

17. **RELATES_TO** - A subplot RELATES_TO the central theme.

18. **TRANSITIONS_FROM** - A scene TRANSITIONS_FROM one setting to another.

19. **INTERACTED_WITH** - A character INTERACTED_WITH another character.

20. **LEADS_INTO** - An event LEADS_INTO the climax.?\
    PARALLEL - story happening in parallel or around the same timeframe

AI Relationships

1. **TRAINED_ON** - A model is TRAINED_ON a dataset.

2. **FINETUNED_FROM** - A model is FINETUNED_FROM a base model.

3. **GENERATED_FROM** - An image is GENERATED_FROM a fine-tuned model.

4. **REQUIRES_DATA** - A model REQUIRES_DATA for training.

5. **BASED_ON** - A remix is BASED_ON a specific workflow.

6. **INFLUENCES** - Sample data INFLUENCES model output.

7. **CREATES** - A pipeline CREATES a fine-tuned model.

8. **UTILIZES** - A workflow UTILIZES a base model.

9. **DERIVED_FROM** - A fine-tuned model is DERIVED_FROM a base model.

10. **PRODUCES** - A model PRODUCES generated images.

11. **MODIFIES** - A remix MODIFIES the base workflow.

12. **REFERENCES** - An AI-generated image REFERENCES original data.

13. **OPTIMIZED_BY** - A model is OPTIMIZED_BY specific algorithms.

14. **INHERITS** - A fine-tuned model INHERITS features from the base model.

15. **APPLIES_TO** - A fine-tuning process APPLIES_TO a model.

16. **COMBINES** - A remix COMBINES elements from multiple datasets.

17. **GENERATES_VARIANTS** - A model GENERATES_VARIANTS of an image.

18. **EXPANDS_ON** - A fine-tuning process EXPANDS_ON base capabilities.

19. **CONFIGURES** - A workflow CONFIGURES a model's parameters.

20. **ADAPTS_TO** - A fine-tuned model ADAPTS_TO new data.

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

Just Governance

> Learn about Story's governance.

As the steward of the Story ecosystem, the Story Foundation works in close alignment with \$IP Tokenholders and the broader ecosystem. The Story Foundation supports the Story DAO by providing operational support, executing tokenholder governance decisions, and overseeing strategic development and growth of the overall ecosystem. This relationship is designed to empower decentralized governance while preserving efficiency and stability throughout the Story ecosystem.

Story DAO's Constitution

<CardGroup cols={1}>
  <Card title="Story DAO Constitution" href="https://story.foundation/constitution.pdf" icon="scroll" color="#ccb092">
    Read the entire Story DAO constitution.
  </Card>
</CardGroup>

Story Foundation's Role in Governance

<Accordion title="Strategic Grants" icon="money-check-dollar">

Provide **strategic grants** to align with innovation via partner projects
including, but not limited to, infrastructure providers, application developers, artists, creators, brand partnerships, creative studios, and
strategic growth partners.
</Accordion>

<Accordion title="Network Security" icon="file-shield">
Promote network security by creating a security council and appointing
members to serve on this council.
</Accordion>

<Accordion title="Implement Proposals" icon="scroll">
Developing the ecosystem and protocol by **implementing proposals** of the
Story DAO that are approved in accordance with the process outlined in the
Story DAO Constitution and engaging parties to build apps. This may include
funding research, public education, and establishing grant programs.
</Accordion>

<Accordion title="Educational Initiatives / Events" icon="book">
Organizing **educational initiatives and hosting events** to increase
awareness of and promote the Story Network, Story Protocol and ecosystem.
</Accordion>

<Accordion title="Decentralization" icon="globe">
Advocating for and **supporting increased autonomy and decentralization** of
the Story DAO.
</Accordion>

<Accordion title="Treasury Management" icon="piggy-bank">
Treasury management and oversight to foster long-term ecosystem growth and
support the Foundation's ongoing mission.
</Accordion>

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

8

📜 Licensing Module

> Learn about creating & attaching real legal license to your IP on Story

<Accordion title="Skip the Read - 1 Minute Summary" icon="circle-info">
The Licensing Module allows you to create a real legal license from a **License Template** (which is the [Programmable IP License (PIL💊)](/concepts/programmable-ip-license/overview)) and attach it to your IP Asset. This license, and the **License Terms** that define it, restrict how others can use your IP, commercialize it, and remix it.

If License Terms are attached to an IP Asset, anyone can mint a **License Token** (an ERC-721 NFT) from it which acts as the license to use that work based on the terms that define it. This token can then be burned to register a derivative work. This then establishes a parent-child relationship between assets, unlocking things like automatic royalty flow from the [💸 Royalty Module](/concepts/royalty-module/overview).
</Accordion>

The owner of an IP Asset owns intellectual property rights such as creating derivatives, being commercially exploited, and being reproduced in different platforms.

IP Assets can programmatically grant permissions for any users to exercise those rights with some autonomy via [License Tokens](/concepts/licensing-module/license-token) (an ERC-721 NFT), which point to a particular set of conditions, known as [License Terms](/concepts/licensing-module/license-terms).

<Frame caption="Blue: contracts built into the protocol. White: contracts developed by the community or 3rd party vendor.">

<img src="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/licensing-module-diagram.png?fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=aef91c68f56292dde370915bfd5645c5" alt="The contracts in blue are built into the protocol. The contracts in white can be developed by the community or 3rd party vendor." data-og-width="2196" width="2196" data-og-height="1054" height="1054" data-path="images/concepts/licensing-module-diagram.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/licensing-module-diagram.png?w=280&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=dc2522c86c0da8dbd8111936f7f7f54d 280w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/licensing-module-diagram.png?w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=06eeb7b718d8faa801b1f503ee0c5a99 560w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/licensing-module-diagram.png?w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=b2183fedd968e4a617622cfb51ca2877 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/licensing-module-diagram.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=f35c0d425453fa6f1c1b2d62f2a00acb 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/licensing-module-diagram.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=78926d4a07c061fd75984e3db3e53039 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/licensing-module-diagram.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=777f414416f9e2904ba35e06c995cd67 2500w" />

</Frame>

LicensingModule

<CardGroup cols={1}>
<Card title="LicensingModule.sol" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/modules/licensing/LicensingModule.sol" icon="scroll" color="#ccb092">
View the smart contract for the License Module.
</Card>
</CardGroup>

The `LicensingModule.sol` contract is the main entry point for the licensing system. It is responsible for:

* Attaching License Terms to IP Assets
* Minting License Tokens
* Registering derivatives
* Setting License Configs

Further Readings

The following document will walk through all of the major components of the Licensing Module as shown above:

* [License Template](/concepts/licensing-module/license-template)
* [License Terms](/concepts/licensing-module/license-terms)
* [License Token](/concepts/licensing-module/license-token)
* [License Registry](/concepts/registry/license-registry)
* [License Config](/concepts/licensing-module/license-config)

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

License Template

> A legal framework, written in code ("programmable"), that defines various licensing terms for an IP

A License Template is a legal framework, written in code ("programmable"), that defines various licensing terms for an IP. Such as:

* "Is commercial use allowed?" - true/false (bool)
* "Is the license transferrable?" - true/false (bool)
* "If commercial, what % of royalty do I receive?" - number

These terms and values differ per License Template.

The first (and currently only) example of a License Template was developed by the Story team directly, and is called the Programmable IP License (PIL :pill:).

<CardGroup cols={2}>
  <Card title="Programmable IP License (PIL)" href="/concepts/programmable-ip-license/overview" icon="pills" color="yellow">
    Learn about the first implementation of a License Template
  </Card>

  <Card title="PIL Smart Contract" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/modules/licensing/PILicenseTemplate.sol" icon="scroll" color="#ccb092">
    View the smart contract for the PIL.
  </Card>
</CardGroup>

License Template Requirements

License Templates are responsible for:

* Providing a link to the actual, off-chain, legal contract template, with all the parameters, their possible values, and the correspondent legalese, in `licenseTextUrl`.
  * For a licensing framework to be compatible with Story, the legal text **must** be clear and parametrized, with each licensing parameter establishing the possible outcomes of each value.
  * The parameter values in each License Template (called "License Template terms") drive the legal text for each license agreement.
* Defining a `struct` with the particular definitions of the parameters in accordance, which must be encoded into the

License Terms struct (described below).
* Providing registration methods for the License Terms, and getters.
* **Verifying** that both the **minter** and the address **linking a derivative are allowed by the License Template terms to perform those actions**.
  * These conditions could be enforced by the License Template itself or through hooks. They can range from limitations on the derivative creations, token-gating LNFT holders, creative control from licensors, KYC, etc. It's up to the implementation of each License Template.
* **Verifying that the License Terms are compatible if a derivative has or will have multiple parents**

Create Your Own Template

You can create your own License Template (like the PIL), but it must be approved by the Story team to be fully embedded into the protocol.

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

License Token

> An ERC-721 NFT that allows you to register your IP as a derivative of another, based on the License Terms defined in the token

<CardGroup cols={1}>
  <Card title="LicenseToken.sol" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/LicenseToken.sol" icon="scroll" color="#ccb092">
    View the smart contract for License Tokens.
  </Card>
</CardGroup>

A **License Token** is represented as an **ERC-721 NFT** and contains the specific [License Terms](/concepts/licensing-module/license-terms) it represents. Its associated `licenseTokenId` is global, as there is one License Token contract.

Once License Terms are attached to an IP Asset, it becomes public so that anyone can mint a License Token for those terms. A License Token is burned when it is used to register another IP as a derivative of the original IP Asset.

<Frame caption="A diagram showing what happens when a License Token is minted.">
  <img src="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-token-diagram.png?fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=de3ecb332bccb1e0ddfd508f758a1aec" alt="A diagram showing what happens when a License Token is minted." data-og-width="2754" width="2754" data-og-height="1268" height="1268" data-path="images/concepts/license-token-diagram.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-token-diagram.png?w=280&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=f8a674c653726ea39e740bfceb6abfae 280w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-token-diagram.png?w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=6d9d496029b7b96e77219e3e87d8a944 560w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-token-diagram.png?

w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=2bfb5533e37c08afb8c1feeedfe20dfc 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-token-diagram.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=9d252b324e4e385fe16940a6f992a05e 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-token-diagram.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=2ae3334ca0b98926577b425c209c6ac1 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-token-diagram.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=a02a14408889dedb420b0f517d0108eb 2500w" />
</Frame>

Private Licenses

<Frame>
<img src="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/private-licenses.png?fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=41fbeab6460274071950c739ed5cbd75" alt="A diagram showing how private licenses are minted." data-og-width="2026" width="2026" data-og-height="1312" height="1312" data-path="images/concepts/private-licenses.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/private-licenses.png?w=280&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=1fa422cc2bad8e99538b4ffdfea2f63e 280w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/private-licenses.png?w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=7552e504d95a122ca4f9e0f407aa0772 560w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/private-licenses.png?w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=e25f1901e325363915ba86eb9556cbea 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/private-licenses.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=ed1215a3c2796c10031d11174ae4b44a 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/private-licenses.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=56686f5f20b862ef9d992e692b8e73e7 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/private-licenses.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=f139e240fec93390df8a1373b736f348 2500w" />
</Frame>

In order to mint a private License Token, the owner of a root IP Asset can issue License Tokens that have terms **not yet attached to the IP Asset itself**. It is important to also note that derivative IP Assets cannot issue private licenses because it is restricted to only issue licenses of its inherited terms.

Transferability of the License Token

License Tokens might be transferrable or not, depending on the values of the License Terms terms they point to.

Once a non-transferable License Token is minted to a recipient, it is locked there forever.

Registering a Derivative

You can register an IP Asset as a derivative of other IP Assets, each with their own license terms agreement. This creates a legally binding agreement between IP Assets that enforces things things like automatic payments in the [💸 Royalty Module](/concepts/royalty-module/overview).

⚠️ Restrictions

There are a few restrictions on registering a derivative:

* An IP Asset can only register as a derivative one time. If an IP Asset has multiple parents, it must register both at the same time.
* Once an IP Asset is a derivative, it cannot link any more parents.
* When you link an IP Asset as a derivative, it cannot have license terms attached. It will inherit its terms from its parents.
* None of the parent IP Assets or the child IP Asset can be disputed.
* The child IP Asset cannot have derivatives already.
* If at least one of the license terms is commercial, then they all must be commercial (`commercialUse = true`)

***

There are two ways to register a derivative IP Asset.

<Note>
  An IP Asset can only register as a derivative one time. If an IP Asset has multiple parents, it must register both at the same time. Once an IP Asset is a derivative, it cannot link any more parents.
</Note>

1. Using an Existing License Token

A License Token is burned when it is used to register another IP as a derivative of the original IP Asset.

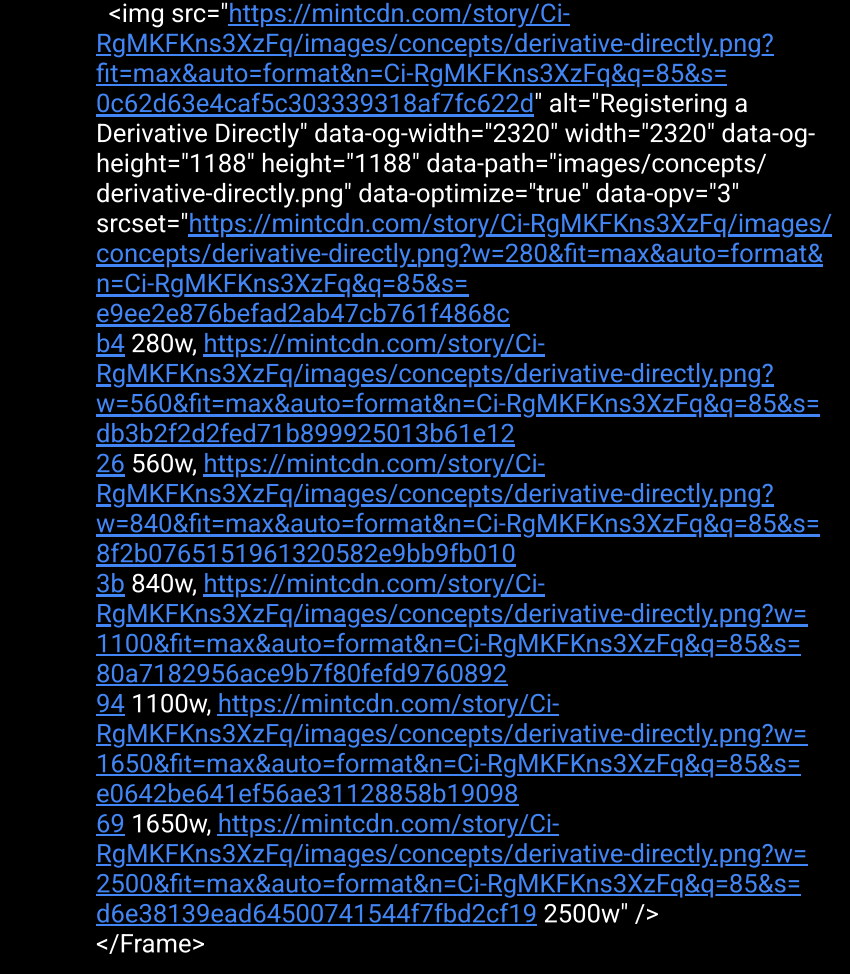
<Frame>

/>
</Frame>

2. Registering a Derivative Directly

You can also register a derivative directly, without the need for a License Token. Remember that if License Terms are attached to an IP Asset it is public to mint the License Token anyway, so this is simply a convenient way to go about it, thus skipping the middle step of minting a License Token.

<Frame>



> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

License Config

> An optional config that can be attached to a specific license for dynamic minting fees and custom logic.

License Config

<CardGroup cols={1}>
  <Card title="LicensingConfig Struct" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/lib/Licensing.sol" icon="scroll" color="#ccb092">
    View the LicensingConfig struct in the smart contract.
  </Card>
</CardGroup>

Optionally, you can attach a `LicensingConfig` to an IP Asset (for a specific `licenseTermsId` attached to that asset) which contains fields like a `mintingFee` and a `licensingHook`, as shown below.

```solidity  theme={null}
/// @notice This struct is used by IP owners to define the configuration
/// when others are minting license tokens of their IP through the LicensingModule.
/// When the `mintLicenseTokens` function of LicensingModule is called, the LicensingModule will read
/// this configuration to determine the minting fee and execute the licensing hook if set.
/// IP owners can set these configurations for each License or set the configuration for the IP
```

```
/// so that the configuration applies to all licenses of the IP.
/// If both the license and IP have the configuration, then the
license configuration takes precedence.
/// @param isSet Whether the configuration is set or not.
/// @param mintingFee The minting fee to be paid when minting
license tokens.
/// @param licensingHook  The hook contract address for the
licensing module, or address(0) if none
/// @param hookData The data to be used by the licensing hook.
/// @param commercialRevShare The commercial revenue share
percentage.
/// @param disabled Whether the license is disabled or not.
/// @param expectMinimumGroupRewardShare The minimum
percentage of the group's reward share
/// (from 0 to 100%, represented as 100 * 10 ** 6) that can be
allocated to the IP when it is added to the group.
/// If the remaining reward share in the group is less than the
minimumGroupRewardShare,
/// the IP cannot be added to the group.
/// @param expectGroupRewardPool The address of the expected
group reward pool.
/// The IP can only be added to a group with this specified reward
pool address,
/// or address(0) if the IP does not want to be added to any group.
struct LicensingConfig {
  bool isSet;
  uint256 mintingFee;
  address licensingHook;
  bytes hookData;
  uint32 commercialRevShare;
  bool disabled;
  uint32 expectMinimumGroupRewardShare;
  address expectGroupRewardPool;
}
```

What do some of these mean?

1. `isSet` - if this is false, the whole licensing config is completely ignored. So for example, if the licensing config has `mintingFee == 10` and `disabled == true`, but the `isSet == false`, the `mintingFee` and `disabled` will be completely ignored.
2. `disabled` - if this is true, then no licenses can be minted and no more derivatives can be attached at all for the terms the config is attached to.

Fields like the `mintingFee` and `commercialRevShare` overwrite their duplicate in the license terms themselves. **A benefit of this is that derivative IP Assets, which normally cannot change their license terms, are able to overwrite certain fields.**

The `licensingHook` is an address to a smart contract that implements the `ILicensingHook` interface, which contains a `beforeMintLicenseTokens` function which will be run before a user mints a License Token. This means you can insert logic to be run upon minting a license.

The hook itself is described in a different section. You can see it contains information about the license, who is minting the License Token, and who is receiving it.

<Tip>
  Learn all about Licensing Hooks [here](/concepts/hooks#licensing-hooks).
</Tip>

Setting the License Config

You can set the License Config by calling the `setLicenseConfig` function in the [LicensingModule.sol contract](https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/modules/licensing/LicensingModule.sol).

Logic that is Possible with License Config

1. **Max Number of Licenses**: The `licensingHook` (described in the next section) is where you can define logic for the max number of licenses that can be minted. For example, reverting the transaction if the max number of licenses has already been minted.
2. **Disallowing Derivatives**: If you register a derivative of an IP Asset, that derivative cannot change its License Terms as described [here](/concepts/licensing-module/license-terms#inherited-license-terms). You can be wondering: "What if I, as a derivative, want to disallow derivatives of myself, but my License Terms allow derivatives and I cannot change this?" To solve this, you can simply set `disabled` to true.
3. **Minting Fee**: Similar to #2 above... what about the minting fee? Although you cannot change License Terms on a derivative IP Asset (and thus the minting fee inside of it), you can change the minting fee for that derivative by modifying the `mintingFee` in the License Config, or returning a `totalMintingFee` from the `licensingHook` (described in the next section).
4. **Commercial Revenue Share**: Similar to #2 and #3 above, you can modify the `commercialRevShare` in the License Config.
5. **Dynamic Pricing for Minting a License Token**: Set dynamic pricing for minting a License Token from an IP Asset based on how many total have been minted, how many licenses the user is minting, or even who the user is. All of this data is available in the `licensingHook` (described in the next section).

... and more.

Restrictions

See [IP Modifications & Restrictions](/concepts/ip-asset/ipa-modifications) for the various restrictions on setting the License Config.

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

License Terms

> A particular combination of values from a License Template that define how others can interact with your IP

When registering your IP on Story, you can attach License Terms to the IP. These are real, legally binding terms enforced on-chain by the [📜 Licensing Module](/concepts/licensing-module/overview), disputable by the [❌ Dispute Module](/concepts/dispute-module/overview), and in the worst case, able to be enforced off-chain in court through traditional means.

In them are also terms for commercial usage, which describes how the [💸 Royalty Module](/concepts/royalty-module/overview) will be enforced (ex. "50% of revenue must be shared with the parent IP").

<CardGroup cols={1}>
  <Card title="Example License Terms" href="/concepts/programmable-ip-license/pil-flavors" icon="ice-cream">
    View some popular combinations of PIL License Terms, also known as
    "flavors".
  </Card>
</CardGroup>

More specifically, License Terms are a particular combination of values from a [License Template](/concepts/licensing-module/license-template). Indeed, there can and will exist **multiple** License Terms (variations) for each License Template. You can imagine that a License Template generates many License Term variations.

<Frame>

<img src="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-diagram.png?fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=5b45745977e3f8097c3d98a7ac636912" alt="License Terms Diagram" data-og-width="1884" width="1884" data-og-height="958" height="958" data-path="images/concepts/license-terms-diagram.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-diagram.png?w=280&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=8c1e7bdc4f05bdd5fe888322ee4fa68d 280w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-diagram.png?w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=3f843652135854b98257bb064f93cb5b 560w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-diagram.png?w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=9ee4594550c5997582e122cfcec99059 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-diagram.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=2454ab4d1cb903d79a4027c91660896a 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-diagram.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=d2bc62444099fc3d59efbf4b8f83f5c7 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-diagram.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=3a2c8649f37d017fa9a677953fb516d0 2500w" />

</Frame>

Once registered, **License Terms are immutable — they can't be tampered with or altered**, even by the License Template that generated it.

Additionally, License Terms have a unique numeric ID within the License Template they stem from. This makes License Terms reusable, meaning if someone creates License Terms with a specific set of values, it only needs to be created once and can be used by anyone else.

For example, a particular set of term values of the [Programmable IP License (PIL💊)](/concepts/programmable-ip-license/overview), such as non-commercial usage + derivatives allowed + free minting, defines a unique License Terms with an associated ID.

License Terms Attached to IP Asset

The owner of a root IP Asset can attach License Terms to signal to other users that they can mint License Tokens of those terms to create a derivative of this IP Asset. **Once License Terms are attached to an IP Asset, it is now considered "public" and anyone can mint a License Token using those terms.**

<Frame>

<img src="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-attach-diagram.png?fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=82a5542f6e65aeece5c64825eb995184" alt="License Terms Attached to IP Asset" data-og-width="1742" width="1742" data-og-height="432" height="432" data-path="images/concepts/license-terms-attach-diagram.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-attach-diagram.png?w=280&fit=max&

auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=eb3dfa5ebf2a9c9da12727662441d3d5 280w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-attach-diagram.png?w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=0cd4a654d1cc4cacf5939b6a944306b0 560w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-attach-diagram.png?w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=1cf654b556a05039d73edcfc1fa56ad0 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-attach-diagram.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=d042930cdefd58166786a5b595171d82 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-attach-diagram.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=b56ae40ce294694d2d1354dd9c2c7ad6 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/license-terms-attach-diagram.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=2261daff029cc4451c85b82f8383a517 2500w" />
</Frame>

Inherited License Terms

On the other hand, derivative IP Assets inherit their License Terms from the parent IP Asset. This means that when an IP Asset registers itself as a derivative, it burns the License Token and inherits the associated License Terms. **The owner of this derivative cannot set new License Terms.**

<Note>
You may be wondering: "if I cannot set new License Terms on my derivative, does that also mean I can't change the minting fee, or disallowing more derivatives, on my derivative?"

Thankfully, there is a way to get around this! Although you cannot change License Terms on a derivative IP, you can utilize the [License Config to implement special behaviors](/concepts/licensing-module/license-config).
</Note>

Expiration

License Terms support an `expiration` time. Once License Terms expire, any derivatives that abide by that license will no longer be able to generate revenue or create further derivatives. If an IP Asset is a derivative of multiple parents, it will expire when the soonest expiration time between the two parents is reached.

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

IP Royalty Vault

> Learn about IP Royalty Vaults and how they manage revenue for IP Assets

An IP Royalty Vault is where all monetary inflows related to an IP Asset are stored. This revenue is then claimed with **Royalty Tokens**.

<Frame>

<img src="https://mintcdn.com/story/KwuxPVrS3UIa7CU1/images/concepts/royalty-module/royalty-vaults.png?fit=max&auto=format&n=KwuxPVrS3UIa7CU1&q=85&s=2470c39509178a39b7524278ea15976c" alt="IP Royalty Vault" data-og-width="988" width="988" data-og-height="1008" height="1008" data-path="images/concepts/royalty-module/royalty-vaults.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/KwuxPVrS3UIa7CU1/images/concepts/royalty-module/royalty-vaults.png?w=280&fit=max&auto=format&n=KwuxPVrS3UIa7CU1&q=85&s=9f66947664fa1a09205f1a4136257854 280w, https://mintcdn.com/story/KwuxPVrS3UIa7CU1/images/concepts/royalty-module/royalty-vaults.png?w=560&fit=max&auto=format&n=KwuxPVrS3UIa7CU1&q=85&s=dc12fd89ce1215280350567598c28c3e 560w, https://mintcdn.com/story/KwuxPVrS3UIa7CU1/images/concepts/royalty-module/royalty-vaults.png?w=840&fit=max&auto=format&n=KwuxPVrS3UIa7CU1&q=85&s=5b1220f8362e8372f13f41c5fb6a6f6f 840w, https://mintcdn.com/story/KwuxPVrS3UIa7CU1/images/concepts/royalty-module/royalty-vaults.png?w=1100&fit=max&auto=format&n=KwuxPVrS3UIa7CU1&q=85&s=35164f6e4b08bb8f36b2f3785e1f9b2e 1100w, https://mintcdn.com/story/KwuxPVrS3UIa7CU1/images/concepts/royalty-module/royalty-vaults.png?w=1650&fit=max&auto=format&n=KwuxPVrS3UIa7CU1&q=85&s=9825ece67f73bfcc5a415dedfd8b6dc5 1650w, https://mintcdn.com/story/KwuxPVrS3UIa7CU1/images/concepts/royalty-module/royalty-vaults.png?w=2500&fit=max&auto=format&n=KwuxPVrS3UIa7CU1&q=85&s=0bf9df102d432097909a9f001356f9e9 2500w" />
</Frame>

<CardGroup cols={1}>
  <Card title="IPRoyaltyVault.sol" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/modules/royalty/policies/IpRoyaltyVault.sol" icon="scroll" color="#ccb092">
    View the smart contract for the IP Royalty Vault.
  </Card>
</CardGroup>

Royalty Tokens

When an IP Asset receives revenue, it is deposited into its IP Royalty Vault. In order to claim revenue from this vault, you must have the associated **Royalty Tokens**. Every vault has 100 Royalty Tokens associated with it. If an address owns these tokens, it is entitled to that % (% of the total supply of Royalty Tokens owned) of any future revenue in the IP Royalty Vault.

The IP Royalty Vault contract is also the ERC-20 contract for the **Royalty Tokens** of each IP Asset. This means the address of the IP Royalty Vault for an IP Asset is also the ERC-20 token address of the Royalty Tokens.

Whitelisted Payment Tokens

An ERC-20 token must be whitelisted by our protocol in the [RoyaltyModule.sol contract](https://github.com/storyprotocol/protocol-core-v1/blob/e339f0671c9172a6699537285e32aa45d4c1b57b/contracts/modules/royalty/RoyaltyModule.sol#L50) to be used for making payments. Here are the whitelisted tokens:

<Tabs>
  <Tab title="Aeneid Testnet">
Token	Contract Address	Explorer	Mint

---------- |
| WIP | `0x1514000000000000000000000000000000000000` | [View here ↗](https://aeneid.storyscan.io/address/0x1514000000000000000000000000000000000000) | N/A |
| MERC20 | `0xF2104833d386a2734a4eB3B8ad6FC6812F29E38E` | [View here ↗](https://aeneid.storyscan.io/address/0xF2104833d386a2734a4eB3B8ad6FC6812F29E38E) | [Mint ↗](https://aeneid.storyscan.io/address/0xF2104833d386a2734a4eB3B8ad6FC6812F29E38E?tab=write_contract#0x40c10f19) |
</Tab>

<Tab title="Mainnet">
Token	Contract Address	Explorer	Mint
WIP	`0x1514000000000000000000000000000000000000`	[View here ↗](https://aeneid.storyscan.io/address/0x1514000000000000000000000000000000000000)	N/A
</Tab>
</Tabs>

How to obtain Royalty Tokens?

There are two ways that trigger the IP Royalty Vault deployment and make the initial Royalty Token distribution - whichever comes first:

1. A License Token is minted from an IP for the first time.
2. A derivative is registered under the IP.

In each case, the associated [IP Account](/concepts/ip-asset/ip-account), which is the IP itself, receives 100% of the Royalty Tokens.

Because Royalty Tokens are ERC-20, they can be transferred like any other token. Thus, the IP Account could send them to someone else, or even put them up for sale on the secondary market. Once they are transferred, the new owner now has the right to claim x% of the revenue from the royalty vault.

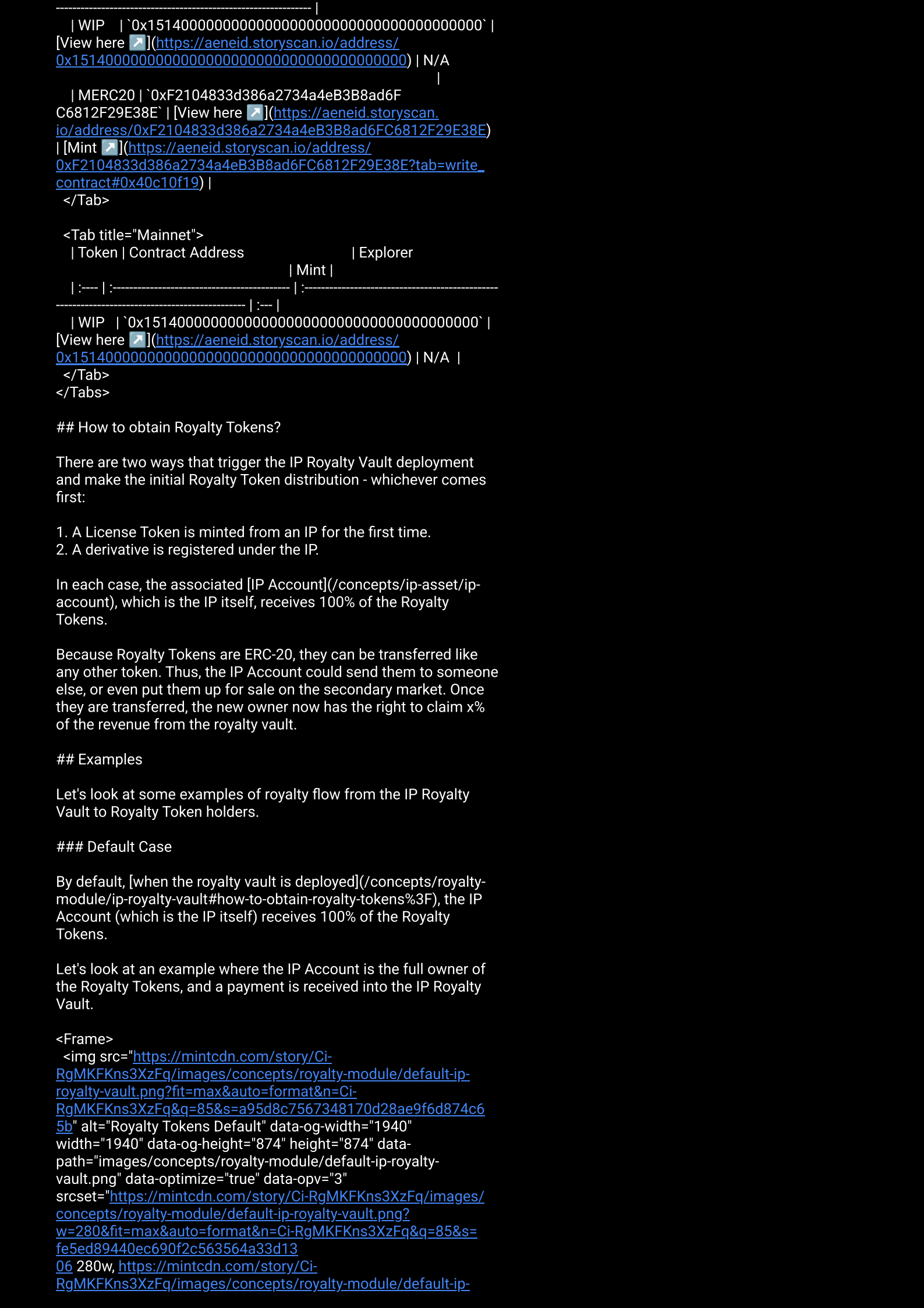
Examples

Let's look at some examples of royalty flow from the IP Royalty Vault to Royalty Token holders.

Default Case

By default, [when the royalty vault is deployed](/concepts/royalty-module/ip-royalty-vault#how-to-obtain-royalty-tokens%3F), the IP Account (which is the IP itself) receives 100% of the Royalty Tokens.

Let's look at an example where the IP Account is the full owner of the Royalty Tokens, and a payment is received into the IP Royalty Vault.

<Frame>


</Frame>

royalty-vault.png?w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=f7089a44d076bdb41a9140dc0d36f2cd 560w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/default-ip-royalty-vault.png?w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=dcbac0e3ab4ca6739a733e0586c855a7 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/default-ip-royalty-vault.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=fd5e10163f41ed56da2174df5fc52626 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/default-ip-royalty-vault.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=af3c890d619d462236e69f5f4f5dfd09 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/default-ip-royalty-vault.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=05ae5b226d89933b5197817ca142ff84 2500w" />
</Frame>

As you can see, each IP claims the full amount from its royalty vault because it has 100% of the royalty tokens.

Multiple Royalty Token Destinations

Now let's look at a case where the IP Asset 2 has distributed 5% of its Royalty Tokens to another address. Maybe this is because they sold them on the secondary market, or maybe it's because they gave them to a friend.

<Frame>

<img src="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/cool-person.png?fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=b130fafa127ad183071c99529867340b" alt="Multiple Royalty Token Destinations" data-og-width="2020" width="2020" data-og-height="1240" height="1240" data-path="images/concepts/royalty-module/cool-person.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/cool-person.png?w=280&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=4d9ab1867ca4e2ba78dfeb7d4614c598 280w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/cool-person.png?w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=20821ac1237559ae669cb819cdb78b6e 560w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/cool-person.png?w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=550d0e38e5b5a0bc4f263d415defaa42 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/cool-person.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=04daa9fa051d56b7e080b708bd4ec4e7 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/cool-person.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=80d082c01409e7d550e26ee7b40b7f52 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/royalty-module/cool-person.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=2854922703bb4fa0d7e2b68e6d0adaf9 2500w" />

</Frame>

As you can see, IP Asset 1 claims its full 10 \$WIP. But IP Asset 2 only claims 9.5 \$WIP. The other 0.5 \$WIP is claimed by the address that owns the 5% of the Royalty Tokens.

Distributing Royalty Tokens

One of the most common questions is: *"Why does royalty end up in the IP Account? Shouldn't it end up in the IP owner's wallet?"*

While the IP Account is the initial owner of 100% of the Royalty Tokens, remember that as the IP owner, you can transfer them to whoever. An easy solution is to transfer 100% of the Royalty Tokens to the IP owner's wallet, so that when revenue is received to the IP Royalty Vault, revenue can be claimed directly to the wallet.

<Card title="Working Code Example" icon="code" href="https://github.com/storyprotocol/typescript-tutorial/blob/main/scripts/royalty/transferRoyaltyTokens.ts">
  View a working code example of transferring Royalty Tokens to an external
  wallet (like the IP owner's wallet). Remember, Royalty Tokens are simple
  ERC-20s!
</Card>

<Frame>

</Frame>

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

Liquid Absolute Percentage (LAP)

> Learn how the Liquid Absolute Percentage royalty policy works.

The Liquid Absolute Percentage (LAP) defines that each parent IP Asset can choose a minimum royalty percentage that **all** of its downstream IP Assets in a derivative chain will share from their monetary gains as defined in the license agreement.

<CardGroup cols={1}>

<Card title="RoyaltyPolicyLAP.sol" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/modules/royalty/policies/LAP/RoyaltyPolicyLAP.sol" icon="scroll" color="#ccb092">
  View the smart contract for the LAP Royalty Policy.
</Card>
</CardGroup>

Calculated Royalty Stack

In the image below, IPA 1 and IPA 2 - due to being ancestors of IPA 3 - have a % economic right over revenue made by IPA 3. Key notes to understand the derivative chain below:

* **License Royalty %**: this percentage is selected by the user and it means the percentage that the user wants - according to LAP rules - in return for allowing other users to remix its IPA.
* **Royalty Stack**: the total revenue % an IPA has to pay to all its ancestors. For LAP, it's the sum of parents royalty stack + sum of licenses percentages used to connect to each parent
  * Royalty Stack IPA 2 = Royalty Stack IPA 1 + License Royalty % between IPAs 1 and 2 = 0% + 5% = 5%
  * Royalty Stack IPA 3 = Royalty Stack IPA 2 + License Royalty % between IPAs 2 and 3 = 5% + 10% = 15%

<Note>
  The "License Royalty %" in this diagram corresponds to the same value as the `commercialRevShare` on the [PIL terms](/concepts/programmable-ip-license/pil-terms).
</Note>

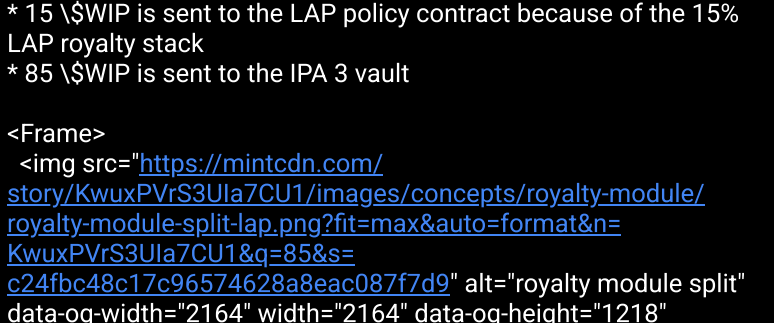
<Frame>

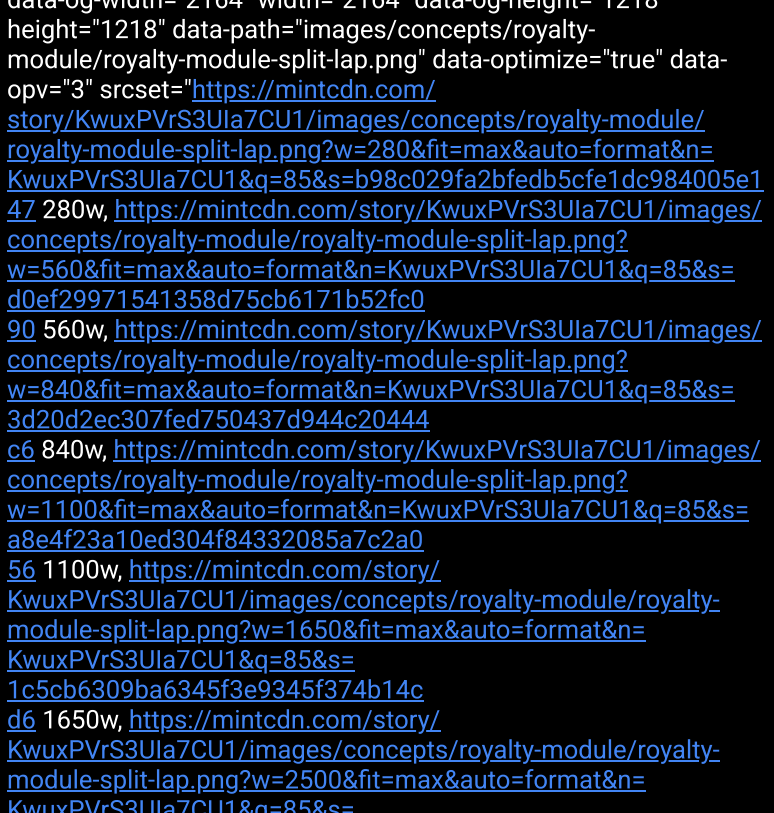
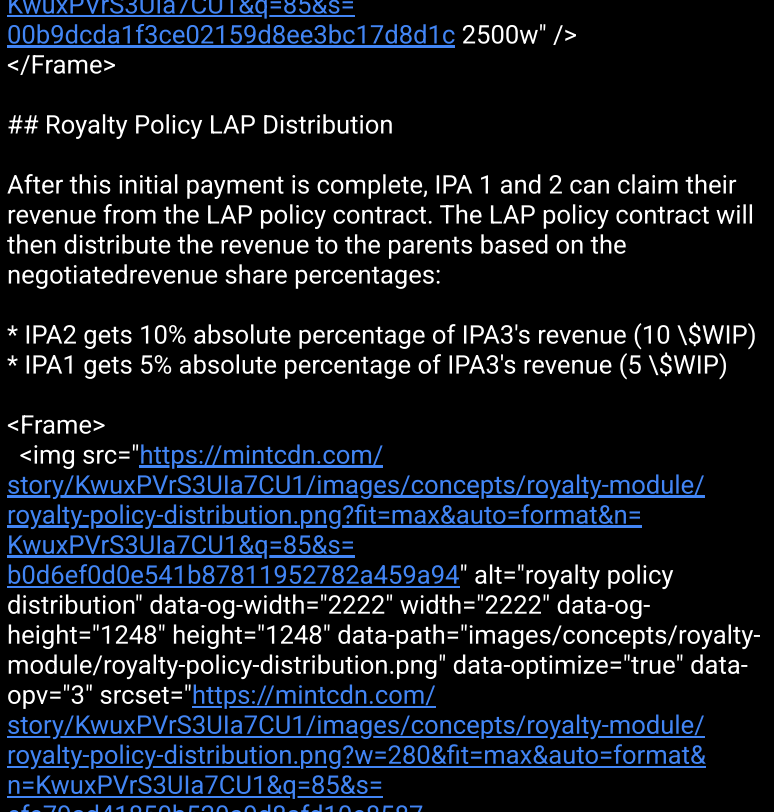
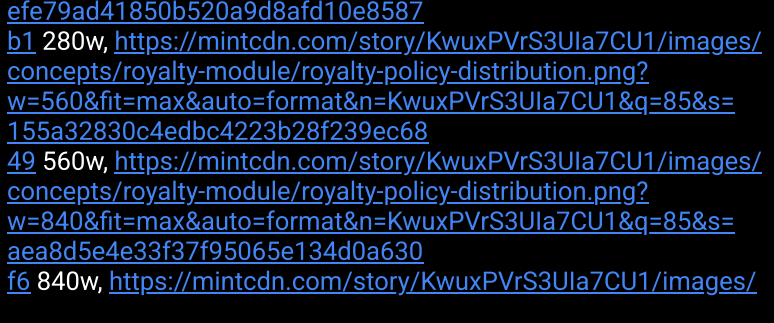
</Frame>

Royalty Module Split

Let's show an example where a payment is made to IPA 3. In the below diagram, you can see that initial payment is forwarded to the Royalty Module. The Royalty Module then splits the payment based on the **Royalty Stack** determined by the LAP policy:

* 15 \$WIP is sent to the LAP policy contract because of the 15% LAP royalty stack
* 85 \$WIP is sent to the IPA 3 vault

<Frame>
  

</Frame>

Royalty Policy LAP Distribution

After this initial payment is complete, IPA 1 and 2 can claim their revenue from the LAP policy contract. The LAP policy contract will then distribute the revenue to the parents based on the negotiatedrevenue share percentages:

* IPA2 gets 10% absolute percentage of IPA3's revenue (10 \$WIP)
* IPA1 gets 5% absolute percentage of IPA3's revenue (5 \$WIP)

<Frame>

</Frame>

concepts/royalty-module/royalty-policy-distribution.png?w=1100&fit=max&auto=format&n=KwuxPVrS3Ula7CU1&q=85&s=ab4807cfdbd1eacfb209f02a22378e0d 1100w, https://mintcdn.com/story/KwuxPVrS3Ula7CU1/images/concepts/royalty-module/royalty-policy-distribution.png?w=1650&fit=max&auto=format&n=KwuxPVrS3Ula7CU1&q=85&s=d386c6b5ac4612ed3a986552d16ae17d 1650w, https://mintcdn.com/story/KwuxPVrS3Ula7CU1/images/concepts/royalty-module/royalty-policy-distribution.png?w=2500&fit=max&auto=format&n=KwuxPVrS3Ula7CU1&q=85&s=e26f9fd4df1bb6a92b925bd2a43a35cd 2500w" />
</Frame>

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

External Royalty Policies

> Learn about custom royalty policies that can be created for specific use cases

There can be many flavors and variations of royalty distribution rules as we observe in the real world. The same can be expected onchain. Whenever a use case requires unique and specific royalty rules, then those set of rules can be registered as an **External Royalty Policy**.

1. What is an External Royalty Policy?

It is a smart contract that inherits a specific interface called `IExternalRoyaltyPolicy`, which defines the view function below:

<CardGroup cols={1}>
  <Card title="IExternalRoyaltyPolicy.sol" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/interfaces/modules/royalty/policies/IExternalRoyaltyPolicy.sol" icon="scroll" color="#ccb092">
    View the smart contract for external royalty policies.
  </Card>
</CardGroup>

```solidity IExternalRoyaltyPolicy.sol theme={null}
/// @notice Returns the amount of royalty tokens required to link a child to a given IP asset
/// @param ipId The ipId of the IP asset
/// @param licensePercent The percentage of the license
/// @return The amount of royalty tokens required to link a child to a given IP asset
function getPolicyRtsRequiredToLink(address ipId, uint32 licensePercent) external view returns (uint32);
```

After developing your smart contract make sure it inherits the interface above and you can register your new External Royalty Policy by calling `registerExternalRoyaltyPolicy` function in [RoyaltyModule.sol](https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/modules/royalty/RoyaltyModule.sol).

2. How does it work?

Let's follow an example of a new External Royalty Policy called "Policy X".

External Royalty Policies are selected by users

An IPA owner decides the royalty policy he/she wants to allow the IP to be remixed with. There are multiple options of royalty rules that can be chosen such as LAP, LRP and other External Royalty Policies. Let's say the user decides to mint a license token with "Policy X". After that, IP2 remixes IP1 and IP3 remixes IP2 and we have the situation as the image below:

<Frame>
  <img src="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-0.png?fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=c6126233d31ef344390db1996019e57d" alt="External Royalty Policy Example" data-og-width="1216" width="1216" data-og-height="714" height="714" data-path="images/concepts/ep-0.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-0.png?w=280&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=8441060a8127a91ad9bc2b94fa9dbe7a 280w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-0.png?w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=0702a64a83a8525179224d16ceacd8f2 560w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-0.png?w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=f63169501ec2caefa8f10ae26f7e5ccf 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-0.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=d5a4a88850d48162d467e26758fc51cb 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-0.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=429054d11b35da81b75430498daaa636 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-0.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=c1c09eb988807326505026d246393d16 2500w" />
</Frame>

Every time there is a remix - the link between the parent and derivative has 2 data points associated:

1. The royalty policy address
   1. "Policy X" address in the example
2. The percentage of royalty tokens the parent demands from derivatives. This percentage can have different meanings depending on the royalty policy being used - ie. it can be a relative percentage, an absolute percentage, an adjusted percentage according to specific rules, etc.
   1. 10% between IP1 and IP2
   2. 50% between IP2 and IP3

External Royalty Policies receive royalty tokens from their users' IPs

Following the example, when each remix is made and during the `onLinkToParents` function call in [RoyaltyModule.sol](https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/modules/royalty/RoyaltyModule.sol), the function `getPolicyRtsRequiredToLink` is called on the "Policy X" address.

```solidity IExternalRoyaltyPolic.sol theme={null}
/// @notice Returns the amount of royalty tokens required to link a child to a given IP asset
/// @param ipId The ipId of the IP asset
/// @param licensePercent The percentage of the license
/// @return The amount of royalty tokens required to link a child to a given IP asset
function getPolicyRtsRequiredToLink(address ipId, uint32 licensePercent) external view returns (uint32);
```

It should return the % of derivative's royalty tokens that the royalty policy demands for the link to happen. That share of royalty tokens are sent to the "Policy X" contract. In the example case:

* "Policy X" receives 3% of RT2 token supply that it can then redistributed to its userbase. IP1 owner wanted 10%, however - let's assume for the sake of the example - that due to the specific use case of "Policy X" and its custom logic, the IP2 owner is granted a special status in the platform in which it it has a 70% discount on the % share it has to give parent IPs due to having a very large distribution network to promote IPs. Therefore, instead of having to give 10% as the license percentage indicated it only gives 3%.
* "Policy X" receives 50% of RT3 token supply that it can then redistributed to its userbase.

<Frame>

<img src="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-1.png?fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=0214065cea64aa60abf99146f88456f5" alt="Royalty Token Distribution" data-og-width="1217" width="1217" data-og-height="715" height="715" data-path="images/concepts/ep-1.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-1.png?w=280&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=55a72a5be005a31c32fa5247e28d4e16 280w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-1.png?w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=6c7bfa9b0360fc624fd2edd04ce42394 560w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-1.png?w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=60ff0630deb0861b663db5f44908d0b9 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-1.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=0113ccfbc487d06d2927142c3d8dbeaa 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-1.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=e8d8d2bd7af5ad2b6f7c3e2ff0a77669 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-1.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=07c80fca0da86bad541975f1d967a24d 2500w" />

</Frame>

External Royalty Policies redistribute value back to their users according to custom rules

There are two ways in which an External Royalty Policy can redistribute value back to its users:

1. Send Royalty Tokens directly to its users
2. Keep the Royalty Tokens in the External Royalty Policy contract and have users claim Revenue Tokens through the said contract

Let's explore both in the context of "Policy X". Let's say that from the 50% of RT3 token supply "Policy X" received - 40% are kept in the "Policy X" contract and 10% are sent to an ancestor royalty vault (IP1).

<Frame>

<img src="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-2.png?fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=8e953bf48efd65cafdb603b312ed1c60" alt="Royalty Token Redistribution" data-og-width="1214" width="1214" data-og-height="714" height="714" data-path="images/concepts/ep-2.png" data-optimize="true" data-opv="3" srcset="https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ep-2.png?w=280&fit=

max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
d4e414d6bc10b9ca2c38f6ceedba90
40 280w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-2.png?
w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
a6d9775b5541deadf0f90d1dcd7003
f8 560w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-2.png?
w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
5ba128a389cf130e46446a2566f1b0
7b 840w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-2.png?
w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
2ffea4f064d6de302b273720f96532
c5 1100w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-2.png?
w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
67597f78d86df10dd271f5866d145f
23 1650w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-2.png?
w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
234e0f946f7e4fe5cea083393f1b3a12 2500w" />
</Frame>

Now let's imagine there is a 1M payment made to IP3 - an example of how the flow would be:

<Frame>
  <img src="https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-3.png?fit=max&
auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
c23716033c65e7a9971336ffab2dc02c" alt="Payment Flow
Example" data-og-width="1218" width="1218" data-og-height="714"
height="714" data-path="images/concepts/ep-3.png" data-
optimize="true" data-opv="3" srcset="https://mintcdn.com/
story/Ci-RgMKFKns3XzFq/images/concepts/ep-3.png?w=280&fit=
max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
8af3bac2d1918955488d6a503f2c89
da 280w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-3.png?
w=560&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
ab2b0eddd2107b42920b1f0cce9c63
68 560w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-3.png?
w=840&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
432fad483a36c233c91fcffb4f475f
f1 840w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-3.png?
w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
11515c3c217fb73c89049b44837218
a9 1100w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-3.png?
w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
edfece97e2efe70af496f15cf80655
ab 1650w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ep-3.png?
w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
aa0e10a73d21449fd3f3acf82709fdb1 2500w" />
</Frame>

From the 1M WIP inflow to IP3 Royalty Vault:

* 500k WIP are claimed by the IP Account 3 which had 50% of RT3 token supply
* 100k WIP are claimed by the IP1 Royalty Vault which has 10% of RT3 token supply via `claimByTokenBatchAsSelf` function
* 400k WIP are claimed by "Policy X" which has 40 of RT3 token supply. This amount is further split by "Policy X" custom contract according to its specific rules - which define y% and z% - to its users.

> ## Documentation Index

> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

👀 Metadata Module

> Manage & view metadata for IP Assets.

The Metadata Module enables the creation, management, and retrieval of metadata for IP Assets within Story. It consists of two main components: the CoreMetadataModule for writing operations and the CoreMetadataViewModule for reading operations.

<CardGroup cols={2}>
  <Card title="CoreMetadataModule.sol" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/modules/metadata/CoreMetadataModule.sol" icon="scroll" color="#ccb092">
    View the smart contract for the Core Metadata Module.
  </Card>

  <Card title="CoreMetadataViewModule.sol" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/modules/metadata/CoreMetadataViewModule.sol" icon="eye" color="#92ccb0">
    View the smart contract for the Core Metadata View Module.
  </Card>
</CardGroup>

Metadata Structure

The metadata for an IP Asset includes:

* **metadataURI**: A URI pointing to the detailed metadata of the IP Asset
* **metadataHash**: A hash of the metadata for verification purposes
* **nftTokenURI**: A URI pointing to the metadata of the NFT associated with the IP Asset
* **nftMetadataHash**: A hash of the NFT metadata for verification
* **registrationDate**: When the IP Asset was registered
* **owner**: The current owner of the IP Asset

CoreMetadataModule (Write Operations)

`CoreMetadataModule.sol` is responsible for writing and updating metadata for IP Assets. It is **stateful** and provides the following key functionalities:

* Setting and updating metadata URIs for IP Assets
* Setting and updating NFT token URIs
* Freezing metadata to make it immutable
* Managing metadata hashes for verification

The module stores metadata in the IP Asset's storage, making it accessible to other modules and applications.

Setting Metadata

To set metadata for an IP Asset, the caller must have appropriate permissions. The CoreMetadataModule provides several functions for setting metadata:

* `setMetadataURI`: Sets just the IP metadataURI and its hash
* `updateNftTokenURI`: Updates the NFT token URI and its hash
* `setAll`: Sets all metadata attributes at once

Here is an example:

```solidity solidity theme={null}
// Set the metadata URI and hash
coreMetadataModule.setMetadataURI(
```

```
    ipAssetAddress,
    "https://example.com/metadata/asset123",
    keccak256("metadata content hash")
);
```

Freezing Metadata

The CoreMetadataModule allows IP Asset owners to freeze metadata, making it immutable. Once frozen, the metadata cannot be changed, ensuring the permanence of the IP Asset's information.

To freeze metadata:

```solidity solidity theme={null}
// Make the metadata immutable
coreMetadataModule.freezeMetadata(ipAssetAddress);
```

You can check if metadata is frozen using:

```solidity solidity theme={null}
// Check if metadata is frozen
bool isFrozen = coreMetadataModule.isMetadataFrozen(
ipAssetAddress);
```

CoreMetadataViewModule (Read Operations)

`CoreMetadataViewModule.sol` is a read-only module that provides access to the metadata stored by the CoreMetadataModule. It follows the View Module pattern and offers these key functionalities:

* Retrieving metadata URIs and hashes
* Retrieving NFT token URIs and metadata hashes
* Generating formatted JSON strings with all metadata attributes
* Checking registration dates and ownership information

Retrieving Metadata

The CoreMetadataViewModule provides various functions to retrieve metadata:

* `getCoreMetadata`: Returns all metadata in a single struct
* `getMetadataURI`: Returns just the metadata URI
* `getNftTokenURI`: Returns the NFT token URI
* `getJsonString`: Returns a formatted JSON string with all metadata

Here is an example:

```solidity solidity theme={null}
// Get the metadata URI
string memory uri = coreMetadataViewModule.
getMetadataURI(ipAssetAddress);

// Get all metadata in one call
CoreMetadata memory metadata = coreMetadataViewModule.
getCoreMetadata(ipAssetAddress);

// Get a JSON representation of all metadata
string memory jsonMetadata = coreMetadataViewModule.
getJsonString(ipAssetAddress);
```

The Metadata Module provides a robust system for managing IP Asset metadata, ensuring that important information about intellectual property is properly recorded, accessible, and can be made immutable when needed.

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

📝 IPA Metadata Standard

> An overview of the IP-specific metadata standard

<Warning>
  We are still figuring out the best way to define an IPA Metadata Standard. For
  the sake of transparency, the following document is our thoughts so far but is
  subject to change as we release future versions.
</Warning>

<CardGroup cols={2}>
  <Card title="Official Ippy IP" href="https://explorer.story.foundation/ipa/0xB1D831271A68Db5c18c8F0B69327446f7C8D0A42" icon="house">
    Check out the official Ippy IP, which has both NFT & IP metadata.
  </Card>

  <Card title="How to Add Metadata to an IP Asset" href="/concepts/ip-asset/overview#nft-vs-ip-metadata" icon="computer">
    Learn how to actually add the IP metadata discussed here to your IP Asset with an explanation or completed code example.
  </Card>
</CardGroup>

This is the JSON metadata that is associated with an IP Asset, and gets stored inside of an IP Account. You must call `setMetadata(...)` inside of the IP Account in order to set the metadata, and then call `metadata()` to read it.

Attributes & Structure

Below are the important attributes you should provide in your IP metadata. Under the **Required For** column is what the specific field is required for:

* 🔍 Story Explorer - this field will help display your IP on the Story Explorer
* 🕵️ [Commercial Infringement Check](/concepts/story-attestation-service) - this field is required if your IP is **commercial** (that is, has `commercialUse = true` license terms attached). We will use these fields to run an infringement check on your IP.
  * See [current limitations](/concepts/story-attestation-service#current-limitations).
* 🤖 AI Agents - used for displaying metadata associated with AI Agents

Property Name	Type	Description	Required For
`title`	`string`	Title of the IP	🔍 Story Explorer
`description`	`string`	Description of the IP	🔍 Story Explorer
`createdAt`	`string`	Date/Time that the IP was created (either ISO8601 or unix format). This field can be used to specify

historical dates that aren't on-chain. For example, Harry Potter was published on June 26. | 🔍 Story Explorer |
`image`	`string`	An image for your IP. **For audio assets, the recommended thumbnail aspect ratio is 1:1. For video assets, it is 16:9.**	🔍 Story Explorer
`imageHash`	`string`	Hash of your `image` using SHA-256 hashing algorithm. See [here](#hashing-content) for how that is done.	🔍 Story Explorer
`creators`	`IpCreator[]`	An array of information about the creators. [See the type defined below](#type-definitions)	🔍 Story Explorer
`mediaUrl`	`string`	Used for infringement checking, points to the actual media (ex. image or audio). **For audio assets, the recommended thumbnail aspect ratio is 1:1. For video assets, it is 16:9.**	🕵️ [Commercial Infringement Check](/concepts/story-attestation-service)
`mediaHash`	`string`	Hashed string of the media using SHA-256 hashing algorithm. See [here](#hashing-content) for how that is done.	🕵️ [Commercial Infringement Check](/concepts/story-attestation-service)
`mediaType`	`string`	Type of media (audio, video, image), based on [mimeType](https://developer.mozilla.org/en-US/docs/Web/HTTP/MIME_types/Common_types). See the allowed media types [here](#media-types).	🕵️ [Commercial Infringement Check](/concepts/story-attestation-service)
`aiMetadata`	`AIMetadata`	Used for registering & displaying AI Agent Metadata. [See the type defined below](#type-definitions)	🤖 AI Agents
N/A	N/A	You can include other values as well.	N/A

Type Definitions

Here are the type definitions for the complex types used in the metadata:

<CodeGroup>
```typescript IpCreator theme={null}
type IpCreator = {
  name: string;
  address: Address;
  contributionPercent: number; // add up to 100
  description?: string;
  image?: string;
  socialMedia?: IpCreatorSocial[];
  role?: string;
};

type IpCreatorSocial = {
  platform: string;
  url: string;
};
```

```typescript AIMetadata theme={null}
type AIMetadata = {
  // this can be any character file you want
  // example: https://github.com/elizaOS/characterfile/blob/main/examples/example.character.json
  characterFileUrl: string;
  characterFileHash: string;
};
```
</CodeGroup>

Media Types

The following media types are allowed for the `mediaType` field:

Media Type	Description
`image/jpeg`	JPEG image
`image/png`	PNG image
`image/apng`	Animated PNG image
`image/avif`	AV1 Image File Format
`image/gif`	GIF image
`image/svg+xml`	SVG image
`image/webp`	WebP image
`audio/wav`	WAV audio
`audio/mpeg`	MP3 audio
`audio/flac`	FLAC audio
`audio/aac`	AAC audio
`audio/ogg`	OGG audio
`audio/mp4`	MP4 audio
`audio/x-aiff`	AIFF audio
`audio/x-ms-wma`	WMA audio
`audio/opus`	Opus audio
`video/mp4`	MP4 video
`video/webm`	WebM video
`video/quicktime`	QuickTime video

Hashing Content

To hash content for the `imageHash` or `mediaHash` fields, you can use the SHA-256 hashing algorithm. Here's an example of how to do this in JavaScript:

<CodeGroup>
```typescript TypeScript theme={null}
import { toHex, Hex } from "viem";

// get hash from a file
async function getFileHash(file: File): Promise<Hex> {
  const arrayBuffer = await file.arrayBuffer();
  const hashBuffer = await crypto.subtle.digest("SHA-256",
arrayBuffer);
  return toHex(new Uint8Array(hashBuffer), { size: 32 });
}

// get hash from a url
async function getHashFromUrl(url: string): Promise<Hex> {
  const response = await axios.get(url, { responseType:
"arraybuffer" });
  const buffer = Buffer.from(response.data);
  return "0x" + createHash("sha256").update(buffer).digest("hex");
}
```

```shell Shell theme={null}
shasum -a 256 myfile.jpg
```
</CodeGroup>

Example Use Cases

<Tabs>
<Tab title="Ippy Mascot">
This is the official Ippy mascot that is registered on mainnet. You can view it on our protocol explorer [here](https://explorer.story.foundation/ipa/0xB1D831271A68Db5c18c8F0B69327446f7C8D0A42).

```json theme={null}
{
  "title": "Ippy",
  "description": "Official mascot of Story.",
  "createdAt": "1728401700",
  "image": "https://ipfs.io/ipfs/QmSamy4zqP91X42k6wS7kLJQVzuYJuW2EN94couPaq82A8",
  "imageHash": "0x21937ba9d821cb0306c7f1a1a2cc5a257509f228ea6abccc9af1a67dd754af6e",
```

"mediaUrl": "https://ipfs.io/ipfs/QmSamy4zqP91X42k6wS7kLJQVzuYJuW2EN94couPaq82A8",
"mediaHash": "0x21937ba9d821cb0306c7f1a1a2cc5a257509f228ea6abccc9af1a67dd754af6e",
"mediaType": "image/png",
"creators": [
{
"name": "Story Foundation",
"address": "0x67ee74EE04A0E6d14Ca6C27428B27F3EFd5CD084",
"description": "The World's IP Blockchain",
"contributionPercent": 100,
"socialMedia": [
{
"platform": "Twitter",
"url": "https://twitter.com/storyprotocol"
},
{
"platform": "Telegram",
"url": "https://t.me/StoryAnnouncements"
},
{
"platform": "Website",
"url": "https://story.foundation"
},
{
"platform": "Discord",
"url": "https://discord.gg/storyprotocol"
},
{
"platform": "YouTube",
"url": "https://youtube.com/@storyFDN"
}
]
}
],
"tags": ["Ippy", "Story", "Story Mascot", "Mascot", "Official"], // experimental field
"ipType": "Character" // experimental field
}
```

</Tab>

<Tab title="Music">
This is an example song generated on [Suno](https://suno.com/) and registered on our testnet. View the below example [on our protocol explorer](https://aeneid.explorer.story.foundation/ipa/0x7d126DB8bdD3bF88d757FC2e99BFE3d77a55509b).

```json  theme={null}
{
"title": "Midnight Marriage",
"description": "This is a house-style song generated on suno.",
"createdAt": "1740005219",
"creators": [
{
"name": "Jacob Tucker",
"address": "0xA2f9Cf1E40D7b03aB81e34BC50f0A8c67B4e9112",
"contributionPercent": 100
}
],
"image": "https://cdn2.suno.ai/image_large_8bcba6bc-3f60-4921-b148-f32a59086a4c.jpeg",
"imageHash": "0xc404730cdcdf7e5e54e8f16bc6687f97c6578a296f4a21b452d8a6ecabd61bcc",
"mediaUrl": "https://cdn1.suno.ai/dcd3076f-3aa5-400b-ba5d-87d30f27c311.mp3",
"mediaHash": "0xb52a44f53b2485ba772bd4857a443e1fb942cf5dda73c870e2d2238ecd607aee",
"mediaType": "audio/mpeg"
}
```

</Tab>
```

<Tab title="AI Agent">
The main difference here is you should supply `aiMetadata` with a character file. You can provide any character file you want, or use [this ElizaOS example](https://github.com/elizaOS/characterfile/blob/main/examples/example.character.json) as a template.

View the below example [on our protocol explorer](https://aeneid.explorer.story.foundation/ipa/0x49614De8b2b02C790708243F268Af50979D568d4).

```json theme={null}
{
  "title": "Story AI Agent",
  "description": "This is an example AI Agent registered on Story.",
  "createdAt": "1740005219",
  "creators": [
    {
      "name": "Jacob Tucker",
      "address": "0xA2f9Cf1E40D7b03aB81e34BC50f0A8c67B4e9112",
      "contributionPercent": 100
    }
  ],
  "image": "https://ipfs.io/ipfs/bafybeigi3k77t5h5aefwpzvx3uiomuavdvqwn5rb5uhd7i7xcq466wvute",
  "imageHash": "0x64ccc40de203f218d16bb90878ecca4338e566ab329bf7be906493ce77b1551a",
  "mediaUrl": "https://ipfs.io/ipfs/bafybeigi3k77t5h5aefwpzvx3uiomuavdvqwn5rb5uhd7i7xcq466wvute",
  "mediaHash": "0x64ccc40de203f218d16bb90878ecca4338e566ab329bf7be906493ce77b1551a",
  "mediaType": "image/webp",
  "aiMetadata": {
    "characterFileUrl": "https://ipfs.io/ipfs/bafkreic6eu4hlnwx46soib62rgkhhmlieko67dggu6bzk7bvtfusqsknfu",
    "characterFileHash": "0x5e253875b6d7e7a4e407da899473b168229def8cc6a783957c35996928494d2d"
  }
}
```
</Tab>
</Tabs>

Optional Properties

The following properties are optional but can provide additional context about your IP Asset:

<Warning>
We are still figuring out the best way to define an IPA Metadata Standard. The
fields below are bound to change or be removed at some point.
</Warning>

Property Name	Type	Description
`ipType`	`string`	Type of the IP Asset, can be defined arbitrarily by the creator. I.e. "character", "chapter", "location", "items", "music", etc
`relationships`	`IpRelationship[]`	The detailed relationship info with the IPA's direct parent asset, such as `APPEARS_IN`, `FINETUNED_FROM`, etc. See more examples [here](#relationship-types).
`watermarkImage`	`string`	A separate image with your watermark already applied. This way apps choosing to use it can render this version of the image (with watermark applied).
`media`	`IpMedia[]`	An array of supporting media. Media type defined below

`app`	`StoryApp`	This is assigned to verified application from Story directly (on a request basis so far). We will map each App ID to a name
`tags`	`string[]`	Any tags that can help surface this IPA
`robotTerms`	`IPRobotTerms`	Allows you to set Do Not Train for a specific agent
N/A	N/A	You can include other values as well.

Type Definitions

<CodeGroup>
```typescript IpRelationship theme={null}
type IpRelationship = {
  parentIpId: Address;
  type: string; // see "Relationship Types" docs below
};
```

```typescript IpMedia theme={null}
type IpMedia = {
  name: string;
  url: string;
  mimeType: string;
};
```

```typescript StoryApp theme={null}
type StoryApp = {
  id: string;
  name: string;
  website: string;
  action?: string;
};
```

```typescript IPRobotTerms theme={null}
type IPRobotTerms = {
  userAgent: string;
  allow: string;
};
```
</CodeGroup>

Relationship Types

The different relationship types that can be used for the `relationships` attribute.

Story Relationships

1. **APPEARS_IN** - A character APPEARS_IN a chapter.

2. **BELONGS_TO** - A chapter BELONGS_TO a book.

3. **PART_OF** - A book is PART_OF a series.

4. **CONTINUES_FROM** - A chapter CONTINUES_FROM the previous one.

5. **LEADS_TO** - An event LEADS_TO a consequence.

6. **FORESHADOWS** - An event FORESHADOWS future developments.

7. **CONFLICTS_WITH** - A character CONFLICTS_WITH another character.

8. **RESULTS_IN** - A decision RESULTS_IN a significant change.

9. **DEPENDS_ON** - A subplot DEPENDS_ON the main plot.

10. **SETS_UP** - A prologue SETS_UP the story.

11. **FOLLOWS_FROM** - A chapter FOLLOWS_FROM the previous one.

12. **REVEALS_THAT** - A twist REVEALS_THAT something unexpected occurred.

13. **DEVELOPS_OVER** - A character DEVELOPS_OVER the course of the story.

14. **INTRODUCES** - A chapter INTRODUCES a new character or element.

15. **RESOLVES_IN** - A conflict RESOLVES_IN a particular outcome.

16. **CONNECTS_TO** - A theme CONNECTS_TO the main narrative.

17. **RELATES_TO** - A subplot RELATES_TO the central theme.

18. **TRANSITIONS_FROM** - A scene TRANSITIONS_FROM one setting to another.

19. **INTERACTED_WITH** - A character INTERACTED_WITH another character.

20. **LEADS_INTO** - An event LEADS_INTO the climax.?\
    PARALLEL - story happening in parallel or around the same timeframe

AI Relationships

1. **TRAINED_ON** - A model is TRAINED_ON a dataset.

2. **FINETUNED_FROM** - A model is FINETUNED_FROM a base model.

3. **GENERATED_FROM** - An image is GENERATED_FROM a fine-tuned model.

4. **REQUIRES_DATA** - A model REQUIRES_DATA for training.

5. **BASED_ON** - A remix is BASED_ON a specific workflow.

6. **INFLUENCES** - Sample data INFLUENCES model output.

7. **CREATES** - A pipeline CREATES a fine-tuned model.

8. **UTILIZES** - A workflow UTILIZES a base model.

9. **DERIVED_FROM** - A fine-tuned model is DERIVED_FROM a base model.

10. **PRODUCES** - A model PRODUCES generated images.

11. **MODIFIES** - A remix MODIFIES the base workflow.

12. **REFERENCES** - An AI-generated image REFERENCES original data.

13. **OPTIMIZED_BY** - A model is OPTIMIZED_BY specific algorithms.

14. **INHERITS** - A fine-tuned model INHERITS features from the base model.

15. **APPLIES_TO** - A fine-tuning process APPLIES_TO a model.

16. **COMBINES** - A remix COMBINES elements from multiple datasets.

17. **GENERATES_VARIANTS** - A model GENERATES_VARIANTS of an image.

18. **EXPANDS_ON** - A fine-tuning process EXPANDS_ON base capabilities.

19. **CONFIGURES** - A workflow CONFIGURES a model's parameters.

20. **ADAPTS_TO** - A fine-tuned model ADAPTS_TO new data.
Io

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

🪝 Hooks

> Add custom logic before minting license tokens or registering derivatives.

Hooks allow developers to create custom implementations, restrictions, and functionality upon minting [License Tokens](/concepts/licensing-module/license-token) or registering derivatives.

There are two types of hooks:

1. **Licensing Hooks**: allow you to [add custom logic before minting license tokens](/concepts/licensing-module/license-config#logic-that-is-possible-with-license-config) (and registering derivatives). For example, requesting a dynamic price, limiting the amount of license tokens that can be minted, whitelists, etc. Licensing Hooks can be added / modified on a licensing config at any point.
2. **Commercializer Checker Hooks**: similar to Licensing Hooks, however they are directly a part of the license terms and do not change. You also cannot return a custom minting fee.

Licensing Hooks

These are contracts that implement the `ILicensingHook` interface, which extends from `IModule`.

Most importantly, a Licensing Hook implements a `beforeMintLicenseTokens` function, which is a function that is called before a License Token is minted to implement [custom logic](/concepts/licensing-module/license-config#logic-that-is-possible-with-license-config) and determine the final `totalMintingFee` of that License Token.

<Note>
  View the `ILicensingHook` smart contract
  [here](https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/interfaces/modules/licensing/ILicensingHook.sol#L26).
</Note>

```solidity ILicensingHook.sol theme={null}
/// @notice This function is called when the LicensingModule mints license tokens.
/// @dev The hook can be used to implement various checks and determine the minting price.
/// The hook should revert if the minting is not allowed.
```

/// @param caller The address of the caller who calling the mintLicenseTokens() function.
/// @param licensorIpId The ID of licensor IP from which issue the license tokens.
/// @param licenseTemplate The address of the license template.
/// @param licenseTermsId The ID of the license terms within the license template,
/// which is used to mint license tokens.
/// @param amount The amount of license tokens to mint.
/// @param receiver The address of the receiver who receive the license tokens.
/// @param hookData The data to be used by the licensing hook.
/// @return totalMintingFee The total minting fee to be paid when minting amount of license tokens.
function beforeMintLicenseTokens(
  address caller,
  address licensorIpId,
  address licenseTemplate,
  uint256 licenseTermsId,
  uint256 amount,
  address receiver,
  bytes calldata hookData
) external returns (uint256 totalMintingFee);
```

Note that it returns the `totalMintingFee`. You may be wondering, "I can set the minting fee in the License Terms, in the `LicenseConfig`, and return a dynamic price from `beforeMintLicenseTokens`. What will the final minting fee actually be?" Here is the priority:

| Minting Fee | Importance |
| --- | --- |
| The `totalMintingFee` returned from `beforeMintLicenseTokens` | Highest Priority |
| The `mintingFee` set in the `LicenseConfig` | ⬇️ |
| The `mintingFee` set in the License Terms | Lowest Priority |

<Warning>
  Beware of potentially malicious implementations of external license hooks.
  Please first verify the code of the hook you choose because it may be not
  reviewed or audited by the Story team.
</Warning>

### Available Hooks

Below are available hooks deployed on our protocol that you can use.

<Info>
  View the deployed addresses for these hooks [here](/developers/deployed-smart-contracts#license-hooks).
</Info>

| Hook | Description | Contract Code |
| --- | --- | --- |
| LockLicenseHook | Stop the minting of license tokens or registering new derivatives. | [View here ↗️](https://github.com/storyprotocol/protocol-periphery-v1/blob/main/contracts/hooks/LockLicenseHook.sol) |
| TotalLicenseTokenLimitHook | Set a limit on the amount of license tokens that can be minted, updatable at any time. | [View here ↗️](https://github.com/storyprotocol/protocol-periphery-v1/blob/main/contracts/hooks/TotalLicenseTokenLimitHook.sol) |

### Implementing the Hooks
```

<CardGroup cols={2}>
  <Card title="SDK Code Example" href="https://github.com/storyprotocol/typescript-tutorial/blob/main/scripts/licenses/oneTimeUseLicense.ts" icon="code">
    A working TypeScript SDK code example that shows how to implement a licensing
    hook. More specifically, how to limit the # of licenses that can be minted.
  </Card>

  <Card title="Solidity Code Example" href="https://github.com/storyprotocol/protocol-periphery-v1/blob/main/test/hooks/TotalLicenseTokenLimitHook.t.sol" icon="code">
    A working Solidity code example that shows how to implement a licensing hook.
    More specifically, how to limit the # of licenses that can be minted.
  </Card>
</CardGroup>

Licensing Hooks are ultimately a smart contract that implements the `ILicensingHook` interface. You can view the interface [here](https://github.com/storyprotocol/protocol-periphery-v1/blob/main/contracts/interfaces/ILicensingHook.sol). We have a few Licensing Hooks deployed already (view the chart above).

In order to actually use a Licensing Hook, you must set it in the Licensing Config, which is basically a set of configurations that you set on License Terms when attaching terms to an IP Asset.

<Steps>
  <Step title="Create Licensing Config">
    First you have to create a Licensing Config:

    ```typescript {6-8} theme={null}
    import { LicensingConfig } from '@story-protocol/core-sdk';

    const licensingConfig: LicensingConfig = {
        isSet: true,
        mintingFee: 0n,
        // address of TotalLicenseTokenLimitHook
        // from https://docs.story.foundation/developers/deployed-smart-contracts
        licensingHook: '0xaBAD364Bfa41230272b08f171E0Ca939bD600478',
        hookData: zeroAddress,
        commercialRevShare: 0,
        disabled: false,
        expectMinimumGroupRewardShare: 0,
        expectGroupRewardPool: zeroAddress,
    }
    ```
  </Step>

  <Step title="Set the Licensing Config">
    Next, we'll set the Licensing Config on the License Terms. In the following example, we'll show this happening upon registering the IP Asset:

    <Tip>
      This code snippet requires a bit of setup, and it meant for developers who
      already understand how to setup the TypeScript SDK. If you want to learn more,
      check out the [working code
      example](https://github.com/storyprotocol/typescript-tutorial/blob/main/scripts/licenses/oneTimeUseLicense.ts).
    </Tip>

    <Note>
      This uses the `registerIpAsset` method found [here](/sdk-reference/ipasset#registeripasset).
    </Note>

```typescript {6-7} theme={null}
const response = await client.ipAsset.registerIpAsset({
    nft: {
        type: 'mint',
        spgNftContract: '0xc32A8a0FF3beDDDa58393d022aF4
33e78739FAbc', // public spg contract for ease-of-use
    },
    licenseTermsData: [
        {
            terms: { defaultMintingFee: 0, commercialUse: true, ... }, //
dummy license terms
            // set the licensing config here
            licensingConfig: licensingConfig
        },
    ],
    ipMetadata: {
        ipMetadataURI: 'test-uri',
        ipMetadataHash: toHex('test-metadata-hash', { size: 32 }),
        nftMetadataHash: toHex('test-nft-metadata-hash', { size: 32
}),
        nftMetadataURI: 'test-nft-uri',
    }
})

console.log(`Token ID: ${response.tokenId}, IPA ID:
${response.ipId}, License Terms ID: ${response.licenseTermsIds}`);
```
</Step>

<Step title="Set the Limit to 1">
Now that we have set the Licensing Config on our terms, we can call the `setTotalLicenseTokenLimit` function on the hook and set the max # of licenses that can be minted to 1.

```typescript  theme={null}
const hookResponse = await client.license.
setMaxLicenseTokens({
  ipId: response.ipId,
  licenseTermsId: response.licenseTermsIds![0],
  maxLicenseTokens: 1000,
});

console.log(`Max license tokens set at transaction hash
${hookResponse.txHash}`);

```
</Step>
</Steps>

Creating a New Licensing Hook

Please follow the below process for creating a new licensing hook and getting it whitelisted into Story's protocol.

1. **Develop your Hook**: You may fork [this template repository](https://github.com/storyprotocol/hook-dev-template) to bootstrap your development. It includes an example hook with tests against our Aeneid protocol, but using it is optional.

<Tip>
See
[license-caller-whitelist-hook](https://github.com/jacob-tucker/license-caller-whitelist-hook)
as an example that was used to develop the
`LicenseCallerWhitelistHook.sol`
hook.
</Tip>

2. **Fork Registered Modules Repository**: Fork the [registered-modules repository](https://github.com/storyprotocol/registered-modules) to your own GitHub account.

3. **Update the Module List**: In your `registered-modules` repository, add your hook's details to the `hook-modules.json` file. Make sure to **deploy & verify on block explorer** your hook on both aeneid and mainnet. Ensure that you adhere to the following JSON structure:

```json theme={null}
{
  "name": "YourModuleName",
  "aeneid": {
    "address": "YourModuleAddress",
    "blockExplorerLink": "YourModuleBlockExplorerLink"
  },
  "mainnet": {
    "address": "YourModuleAddress",
    "blockExplorerLink": "YourModuleBlockExplorerLink"
  }
}
```

Replace `YourModuleName`, `YourModuleAddress`, and `YourModuleBlockExplorerLink` with your hook's name, its address, and the link to its block explorer page, respectively.

Example:

```json theme={null}
{
  "name": "LicenseCallerWhitelistHook",
  "aeneid": {
    "address": "0x37be56d9fb06d885cda3cb010096c94c28b4d658",
    "blockExplorerLink": "https://aeneid.storyscan.io/address/0x37be56d9fb06d885cda3cb010096c94c28b4d658?tab=contract"
  },
  "mainnet": {
    "address": "0x6d9d51a444c8318e8840e75dab7ed81b5a714610",
    "blockExplorerLink": "https://www.storyscan.io/address/0x6d9d51a444c8318e8840e75dab7ed81b5a714610?tab=contract"
  }
}
```

4. **Create a Pull Request (PR)**: Once you have added your hook, create a pull request against this repository. In your PR's description, add the following information (replace the values with your own):

```md theme={null}
## Register my module

- Module type: `hook`
- Module name: `LicenseCallerWhitelistHook`
- Aeneid Module address: `0x37be56d9fb06d885cda3cb010096c94c28b4d658`
- Mainnet Module address: `0x6d9d51a444c8318e8840e75dab7ed81b5a714610`
- My module is immutable: yes
- My module is using an upgradeable proxy: no
- My module has been verified on the block explorer (required): yes
- Summary of my module: Hook for allowing a licensor to gate which addresses can mint a license. The licensor can add/remove an address at any time.
- GitHub repository with source code and tests: https://github.com/jacob-tucker/license-caller-whitelist-hook
```

5. **Await Verification**: After your PR is submitted, it will be reviewed. Once a security audit has been performed and completed, and the module whitelisted into the protocol, the PR will be merged. At this point your module will be officially

registered and recognized as safe for use within the Story community.

We look forward to seeing your contributions and expanding the Story module ecosystem!

Commercializer Checker Hooks

<Warning>
  Documentation coming soon. If you have questions in the meantime, ask in the [Builder's Discord](https://discord.gg/storybuilders).
</Warning>

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

🗂️ Registry

> Learn about the various registries that maintain the global state of Story's protocol.

The various registries on Story function as a primary directory/storage for the global states of the protocol. Obviously, they also contain functions to update that storage.

Unlike [⚙️ IP Accounts](/concepts/ip-asset/ip-account), which manage the state of specific IPs, a **registry** oversees the broader states of the protocol.

Types of Registries

Below are all of the registries on Story.

[IP Asset Registry](/concepts/registry/ip-asset-registry)

Responsible for registering IPs into the protocol.

[Group IP Asset Registry](/concepts/registry/group-ip-asset-registry)

Responsible for registering and maintaining Group IP Assets.

[License Registry](/concepts/registry/license-registry)

Stores all license-related states within the protocol, like attaching License Terms to IP Assets, registering derivatives, creating new License Templates, etc.

[Module Registry](/concepts/registry/module-registry)

Maintains and updates the global list of modules and hooks registered permissionlessly on Story

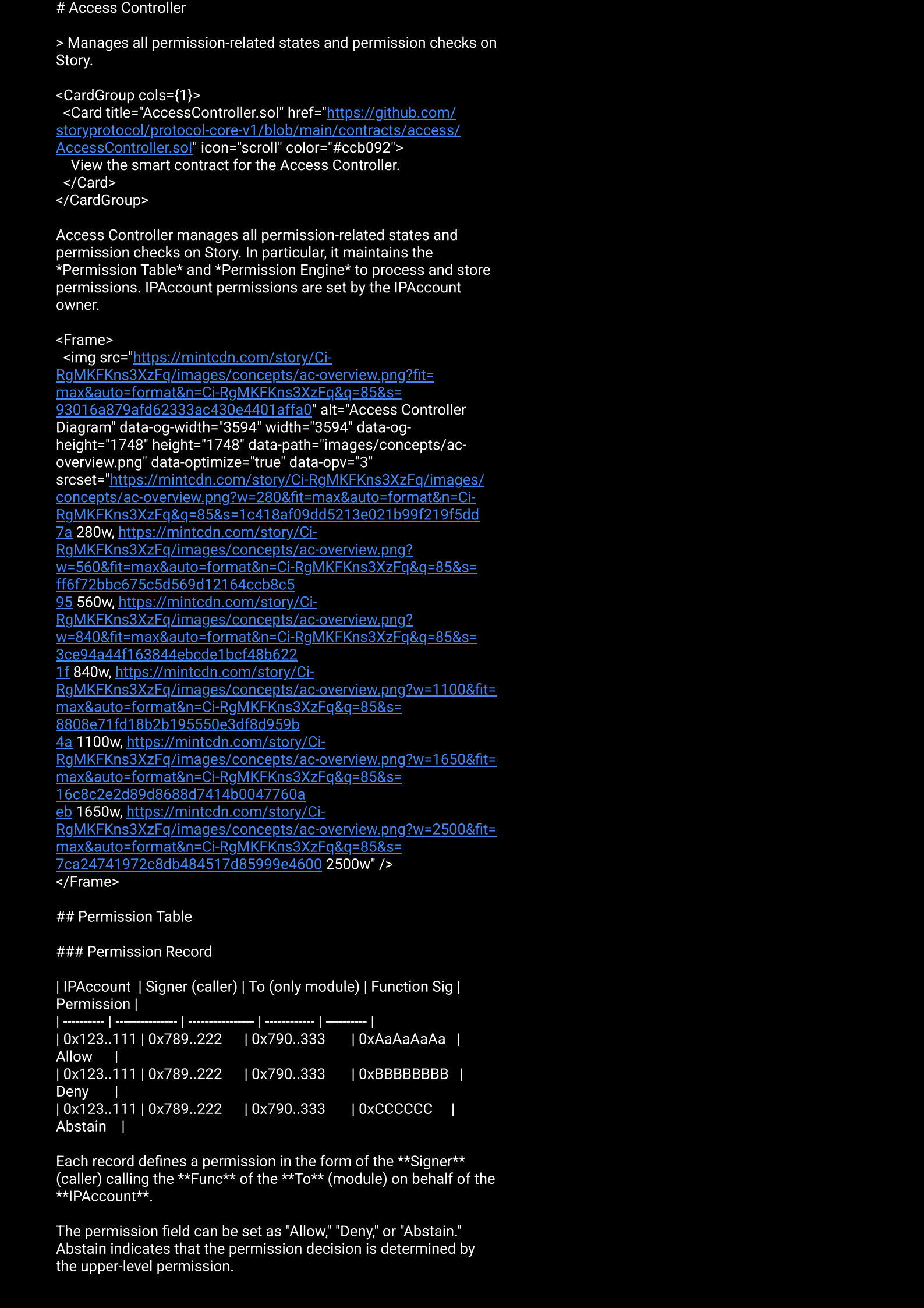
> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

Access Controller

> Manages all permission-related states and permission checks on Story.

<CardGroup cols={1}>
  <Card title="AccessController.sol" href="https://github.com/storyprotocol/protocol-core-v1/blob/main/contracts/access/AccessController.sol" icon="scroll" color="#ccb092">
    View the smart contract for the Access Controller.
  </Card>
</CardGroup>

Access Controller manages all permission-related states and permission checks on Story. In particular, it maintains the *Permission Table* and *Permission Engine* to process and store permissions. IPAccount permissions are set by the IPAccount owner.

<Frame>
  

</Frame>

Permission Table

Permission Record

IPAccount	Signer (caller)	To (only module)	Function Sig	Permission
0x123..111	0x789..222	0x790..333	0xAaAaAaAa	Allow
0x123..111	0x789..222	0x790..333	0xBBBBBBBB	Deny
0x123..111	0x789..222	0x790..333	0xCCCCCC	Abstain

Each record defines a permission in the form of the **Signer** (caller) calling the **Func** of the **To** (module) on behalf of the **IPAccount**.

The permission field can be set as "Allow," "Deny," or "Abstain." Abstain indicates that the permission decision is determined by the upper-level permission.

Wildcard

Wildcard is also supported when defining permissions; it defines a permission that applies to multiple modules and/or functions.

With wildcards, users can easily define a whitelist or blacklist of permissions.

IPAccount	Signer (caller)	To (module)	Func	Permission
0x123..111	0x789..222	*	*	Allow
0x123..111	0x789..222	0x790..333	*	Deny

The above example shows that the signer (0x789...) is unable to invoke any functions of the module (0x790...) on behalf of the IPAccount (0x123...).

In other words, the IPAccount has blacklisted the signer from calling any functions on the module 0x790...333

* Supported wildcards:

Parameter	Wildcard
Func	bytes4(0)
Addresses (IPAccount / To)	address(0)

Permission Prioritization

Specific permissions override general permissions.

IPAccount	Signer (caller)	To (module)	Func	Permission
0x123..111	0x789..222	*	*	Allow
0x123..111	0x789..222	0x790..333	*	Deny
0x123..111	0x789..222	0x790..333	0xCCCCDDDD	Allow

The above shows that the signer (0x789...) is not allowed to call any functions of the module (0x790...) on behalf of IPAccount (0x123...), except for the function 0xCCCCDDDD

Furthermore, the signer (0x789...) is permitted to call all other modules on behalf of IPAccount (0x123...).

Call Flows with Access Control

There exist three types of call flows expected by the Access Controller.

1. An IPAccount calls a module directly.
2. A module calls another module directly.
3. A module calls a registry directly.

IPAccount calling a Module directly

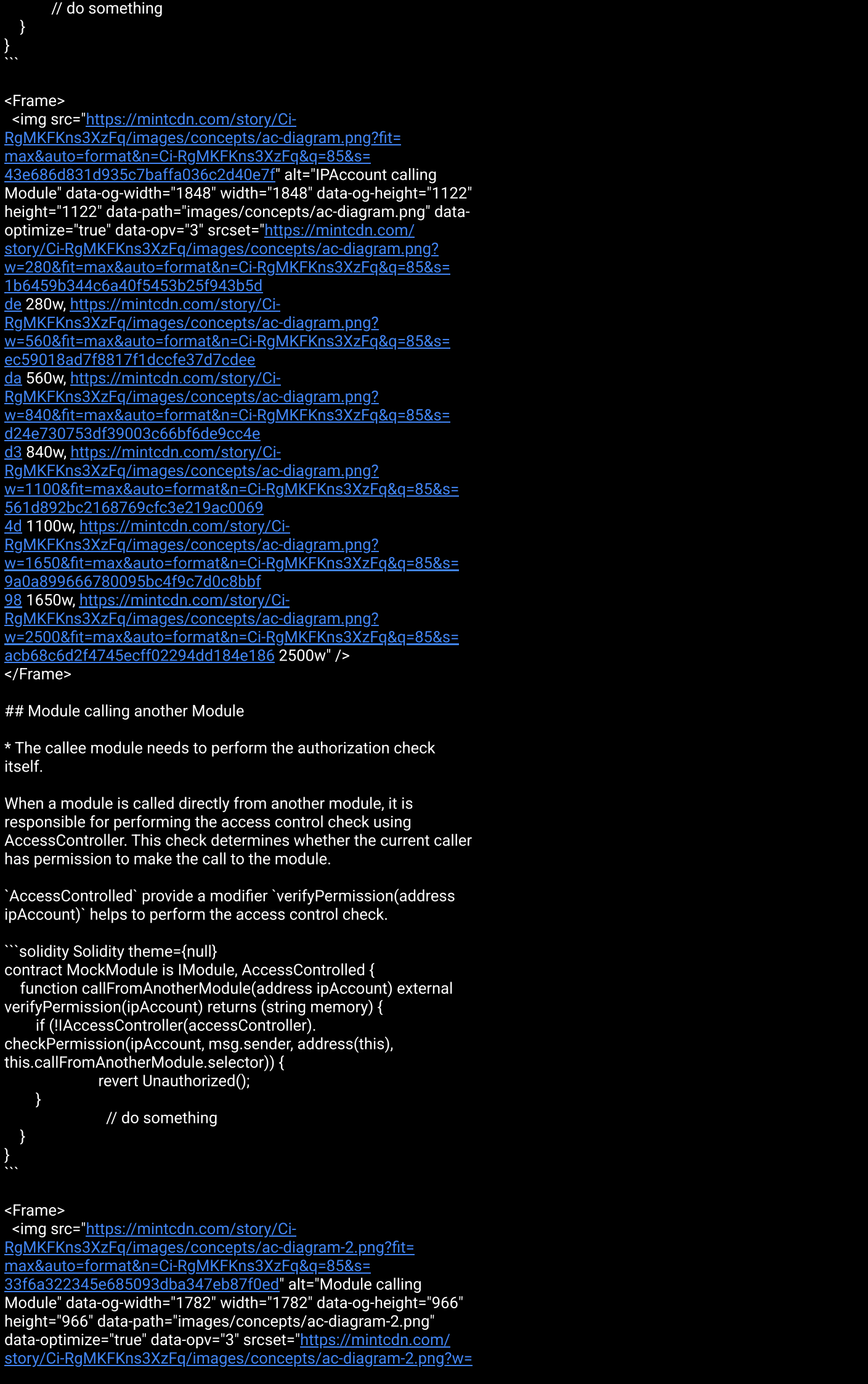
* IPAccount performs a permission check with the Access Controller.
* The module only needs to check if the `msg.sender` is a valid IPAccount.

When calling a module from an IPAccount, the IPAccount performs an access control check with AccessController to determine if the current caller has permission to make the call. In the module, it only needs to check whether the transaction `msg.sender` is a valid IPAccount.

`AccessControlled` provide a modifier `onlyIpAccount()` helps to perform the access control check.

```solidity Solidity theme={null}
contract MockModule is IModule, AccessControlled {
    function action(string memory param) external view
onlyIpAccount() returns (string memory) {
```

```
        // do something
    }
}
```

<Frame>


</Frame>

Module calling another Module

* The callee module needs to perform the authorization check itself.

When a module is called directly from another module, it is responsible for performing the access control check using AccessController. This check determines whether the current caller has permission to make the call to the module.

`AccessControlled` provide a modifier `verifyPermission(address ipAccount)` helps to perform the access control check.

```solidity Solidity theme={null}
contract MockModule is IModule, AccessControlled {
    function callFromAnotherModule(address ipAccount) external verifyPermission(ipAccount) returns (string memory) {
        if (!IAccessController(accessController).checkPermission(ipAccount, msg.sender, address(this), this.callFromAnotherModule.selector)) {
                    revert Unauthorized();
        }
                    // do something
    }
}
```

<Frame>

</Frame>

280&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
40d7df2d70ec43f1c757033e1f6228
9e 280w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ac-diagram-2.png?w=560&fit=
max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
7b14f2876f373244c06d237e0a74c6
25 560w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ac-diagram-2.png?w=840&fit=
max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
9c75981e740b614b44dce2e06b41f2
d0 840w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ac-diagram-2.png?
w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
a27801b3fc334425d232c832f61b30
b7 1100w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ac-diagram-2.png?
w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
ec7a3745a2f6c48e71f2ddfb640c44
12 1650w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ac-diagram-2.png?
w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
b2e79d64442b674352a9a4bfa9bc157b 2500w" />
</Frame>

Module calling Registry

* The registry performs the authorization check by calling AccessController.
* The registry authorizes modules through set global permission

When a registry is called by a module, it can perform the access control check using AccessController. This check determines whether the callee module has permission to call the registry.

```solidity Solidity theme={null}
// called by StoryProtocl Admin
IAccessController(accessController).
setGlobalPermission(address(0), address(module),
address(registry), bytes4(0))) {

```

```solidity Solidity theme={null}
contract MockRegistry {
    function registerAction() external returns (string memory) {
        if (!IAccessController(accessController).
checkPermission(address(0), msg.sender, address(this),
this.registerAction.selector)) {
                    revert Unauthorized();
        }
                    // do something
    }
}
```

<Frame>
  <img src="https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ac-diagram-3.png?fit=
max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
99d22cec31c03de8a9225867867747e7" alt="Module calling
Registry" data-og-width="1776" width="1776" data-og-height="964"
height="964" data-path="images/concepts/ac-diagram-3.png"
data-optimize="true" data-opv="3" srcset="https://mintcdn.com/
story/Ci-RgMKFKns3XzFq/images/concepts/ac-diagram-3.png?w=
280&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
f4c0099d44b06b188cdf728bbe574d
04 280w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ac-diagram-3.png?w=560&fit=
max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
d912037e3d4eb0522059d41907c8aa
75 560w, https://mintcdn.com/story/Ci-
RgMKFKns3XzFq/images/concepts/ac-diagram-3.png?w=840&fit=
max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=
1e07388242710a10cdf4e32cdc9513

85 840w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ac-diagram-3.png?w=1100&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=00360e6bf54d3885b9b580c0afe1576d 1100w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ac-diagram-3.png?w=1650&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=70a4090ed869f5e3e3fca889cc311e86 1650w, https://mintcdn.com/story/Ci-RgMKFKns3XzFq/images/concepts/ac-diagram-3.png?w=2500&fit=max&auto=format&n=Ci-RgMKFKns3XzFq&q=85&s=04daa01d2e28c18f4237bbc05150128b 2500w" />
</Frame>

<Note>
The IPAccount's permissions will be revoked upon transfer of ownership.

The permissions associated with the IPAccount are exclusively linked to its current owner. When the ownership of the IPAccount is transferred to a new individual, the existing permissions granted to the previous owner are automatically revoked. This ensures that only the current, legitimate owner has access to these permissions. If, in the future, the IPAccount ownership is transferred back to the original owner, the permissions that were initially revoked will be reinstated, restoring the original owner's access and control.
</Note>

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

📦 SPG (Periphery)

> Learn about the Story Protocol Gateway that simplifies interactions with the protocol

The Story Protocol Gateway (SPG) is a group of periphery/utility smart contracts, deployed on our protocol that **allows you to combine independent operations** - like registering an [🧩 IP Asset](/concepts/ip-asset/overview) and attaching License Terms to that IP Asset - **into one transaction to make your life easier**.

This was primarily developed to make our [SDK](/sdk-reference) easier to use.

For example, this `mintAndRegisterIpAndAttachPILTerms` is one of the functions in the SPG (more specifically in the `LicenseAttachmentWorkflows.sol`) that allows you to mint an NFT, register it as an IP Asset, and attach License Terms to it all in one call:

```solidity LicenseAttachmentWorkflows.sol theme={null}
function mintAndRegisterIpAndAttachPILTerms(
  address spgNftContract,
  address recipient,
  WorkflowStructs.IPMetadata calldata ipMetadata,
  WorkflowStructs.LicenseTermsData[] calldata licenseTermsData,
  bool allowDuplicates
) external onlyMintAuthorized(spgNftContract) returns (address ipId, uint256 tokenId, uint256[] memory licenseTermsIds)
```

All Supported Workflows

As mentioned above, there are many different functions we have created for you that combine multiple functions into one. We have

categorized them into different groups. These groups are called "workflows".

<CardGroup cols={2}>
  <Card title="View all Workflows" href="https://github.com/storyprotocol/protocol-periphery-v1/blob/main/docs/WORKFLOWS.md" icon="eyes" color="grey">
    Click here to view all of the supported workflows.
  </Card>

  <Card title="Smart Contracts" href="https://github.com/storyprotocol/protocol-periphery-v1/tree/main/contracts/workflows" icon="scroll" color="#ccb092">
    Click here to view the workflow smart contracts.
  </Card>
</CardGroup>

Batching Calls

Although the SPG contains certain functions like `mintAndRegisterIpAndAttachPILTerms`, `registerIpAndAttachPILTerms`, and a bunch more, it would be tedious for us to continually update the contract to account for every single combination of possible interactions with an IP Asset.

Instead, we have allowed for a "Multicall" mechanism where you can batch transactions how you like. For more info, see [Batch Function Calls](/concepts/spg/batch-spg-function-calls).

> ## Documentation Index
> Fetch the complete documentation index at: https://docs.story.foundation/llms.txt
> Use this file to discover all available pages before exploring further.

Story Attestation Service

> A multi-layered decentralized approach to validating intellectual property.

<Accordion title="Skip the Read - 1 Minute Summary" icon="circle-info">
  You can think of the Story Attestation Service (SAS) as a bunch of independent service providers each proving the validity of an IP in their own way. So that each IP has a set of "badges" on it displaying the results.

  It's then up to the ecosystem/market to determine which providers they trust or want to believe. This becomes a decentralized "validator"-like approach to IP validity, where if an IP Asset has lots of providers saying it is valid, then it's probably valid.
</Accordion>

Story employs a multi-layered decentralized approach to validating intellectual property, grounded in two foundational components:

1. The Story Attestation Service (SAS): leverages a network of specialized service providers — each detecting copyright violations across different mediums (images, audio, etc) — to provide transparent, publicly accessible signals on the legitimacy of an [🧩 IP Asset](/concepts/ip-asset). Applications that facilitate IP registration (e.g. original content) may also attest to the provenance of an IP asset (called "apptestations") in the future.
2. The [❌ Dispute Module](/concepts/dispute-module): offers a flexible framework for resolving conflicts, tapping both on-chain and off-chain processes to accommodate the nuanced nature of IP disputes.

This blend of detection methods and dispute resolution creates a robust ecosystem that allows IP to be registered without introducing undue friction, while letting the market, and individual ecosystem apps, determine how much weight to give each attestation provider.

These layers make up the **IP Validation Service (IPVS)** - a fully decentralized marketplace of trust. The existing system of detection providers will continue to expand into a broader ecosystem of signal contributors, each able to offer specialized, verifiable assessments of IP authenticity. Through incentivized participation, IPVS fosters a self-sustaining market where different validators collaborate to deliver specialized signals.

So rather than preventing duplicates, which would cause far more potentially disruptive front running risk, the signals and attestations allow the original IPs surface above the rest.

"When does the SAS scan my IP?"

<Note>
It's important to note that the Story Attestation Service only runs IP infringement checks on **commercial IP**. That is, IP Assets who have at least one [License Terms](/concepts/licensing-module/license-terms) where `commercialUse = true`.

If your IP is non-commercial, then this section doesn't apply to you.
</Note>

When [registering your IP on Story](/developers/tutorials/how-to-register-ip-on-story), you pass in IP-specific metadata that implements the [📝 IPA Metadata Standard](/concepts/ip-asset/ipa-metadata-standard). In this standard, you'll see 3 fields:

Property Name	Type	Description
`mediaUrl`	`string`	Used for infringement checking, points to the actual media (ex. image or audio)
`mediaHash`	`string`	Hashed string of the media using SHA-256 hashing algorithm. See [here](/concepts/ip-asset/ipa-metadata-standard#hashing-content) for how that is done.
`mediaType`	`string`	Type of media (audio, video, image), based on [mimeType](https://developer.mozilla.org/en-US/docs/Web/HTTP/MIME_types/Common_types). See the allowed media types [here](/concepts/ip-asset/ipa-metadata-standard#media-types).

These are used for the commercial infringement check. Whatever media you pass in through `mediaUrl` will be checked by our infringement detection providers and flagged if infringement is detected.

If you do not pass in these `media.*` fields, then an infringement detection will not be performed and your IP will not be proven valid.

Current Limitations

* You must set the `media.*` fields before attaching commercial terms (`commercialUse = true`), otherwise no check will be performed.
* Attestations will only show up on the IP Portal (our "GitHub for IP" platform coming soon). We are working on publishing attestations to public record so anyone can access the results (**COMING SOON!**).
* Only media that is **existing on the internet** will be detected. If someone registers new IP on Story, it will simply return validated

because our providers don't have data on it.

Current Providers

<CardGroup cols={2}>
  <Card title="Yakoa" href="https://www.yakoa.io/" icon="house" color="#190087">
    Yakoa uses AI and machine learning to scan multiple blockchains, analyzing on-chain data to detect direct copies, stylistic forgeries, and unauthorized replications of digital assets. It compares new assets against a database of known IP, flagging potential violations in real time and providing detailed audit logs for enforcement.
  </Card>

  <Card title="Pex" href="https://www.pex.com/" icon="house" color="#019cf4">
    Pex.com is a digital platform that leverages advanced content recognition and analytics to help creators and rights holders track, manage, and monetize their visual and audio media online. It monitors how content is used across the web, making it easier for users to discover licensing opportunities and protect their intellectual property.
  </Card>
</CardGroup>

Becoming an Attestation Provider

The Story Attestation Service is undergoing active development. If you run any form of IP validation (infringement, identity, origin, etc), then you can become an attestation provider. To do so, please fill out this [form](https://docs.google.com/forms/d/10n3AnWoiLsxpaY17kJlxRazysDe8aOWJgirRnfkFRAk/edit).